LaburAR v0.2.6

Plataforma de Intermediación Laboral Local

Autor: Nicolas Menin

Fecha: Octubre 9 2026

Localidad: Punta Alta, Buenos Aires

ÍNDICE

1. **INTRODUCCIÓN**.....6
   1.1. Descripción de proyecto.....6
   1.2. Problemática y justificación.....6
   1.3. Alcance del sistema.....6
2. **ENTREVISTAS**.....7
   2.1. Síntesis de los resultados.....8
      2.1.1. Entrevista 1 Panadería “La Tauriña” Sede 1.....8-9
      2.1.2. Entrevista 2 Panadería “La Tauriña” Sede 2.....9-10
      2.1.3. Entrevista 3 Perfumeria “Hiperfumerias”.....10
      2.1.4. Entrevista 4 Tienda de ropa “ Romina Escoba”.....10-11
   2.2. Necesidades y problemáticas identificadas.....11-12
3. **REQUERIMIENTOS**.....13
   3.1. Requerimientos funcionales.....13-14
   3.2. Requerimientos no funcionales.....14-15
4. **ANÁLISIS DEL SISTEMA**.....16
   4.1. Perfiles de usuarios.....16
      4.1.1. Postulante.....16
      4.1.2. Empleador.....16
      4.1.3. Administrador.....16
   4.2. Diagrama Contextual.....17
   4.3. Tabla de eventos.....18-19
   4.4. Casos de uso.....20
      4.4.1. DCU-General.....20
      4.4.2. DCU-Postulante.....21
      4.4.3. DCU-Empleador.....22
      4.4.4. DCU-Administrador.....23
5. **DISEÑO DEL SISTEMA**.....24
   5.1. Arquitectura del sistema.....24
   5.2. Diseño visual.....25
   5.3. Interfaz de acceso y generales.....26
      5.3.1. Interfaz de selección de usuario.....26
      5.3.2. Interfaz de registro.....26-27
      5.3.3. Interfaz de inicio de sesión.....27
      5.3.4. Interfaz de recuperación de contraseña..... 28
   5.4. Módulo de postulante.....29
      5.4.1. Perfil personal.....29
      5.4.2. Interfaz de postulaciones.....30
      5.4.3. Solicitudes de contacto.....30

5.5. Módulo de empleador.....31
5.5.1. Perfil personal.....31
5.5.2. Gestión de ofertas.....32
5.5.3. Gestión de postulantes.....32
5.5.4. Gestión de solicitudes de contacto.....33
5.6. Módulo de administración.....33
5.6.1. Perfil personal.....34
5.6.2. Gestión de usuarios.....34
5.6.3. Gestión de reportes.....35
5.7. Funcionalidades Transversales.....36
5.7.1. Interfaz de búsqueda.....36
5.7.2. Interfaz de mensajería.....36
5.7.3. Interfaz de reportes.....37
5.7.4. Visualización de perfiles.....37-38
6. **BASE DE DATOS**.....39
6.1. Diagrama entidad-relación (Conceptual).....40
6.2. Esquema de datos relacional (Lógico).....41
6.3. Entidades y tablas.....42
6.3.1. Usuarios.....42
6.3.2. Postulantes.....42
6.3.3. Empleadores.....43
6.3.4. Administradores.....43
6.3.5. Habilidades.....43
6.3.6. Estudios.....44
6.3.7. Experiencias Laborales.....44
6.3.8. Ofertas Laborales.....45
6.3.9. Postulaciones.....46
6.3.10. Conexiones Laborales.....46
6.3.11. Conversaciones.....46
6.3.12. Mensajes.....47
6.3.13. Reportes.....47
6.4. Relaciones entre tablas..... 48
6.4.1. Relaciones de Usuarios.....48-49
6.4.2. Relaciones de Postulantes.....50-51
6.4.3. Relaciones de Empleadores.....51-52
6.4.4. Relaciones de OfertasLaborales.....52
6.4.5. Relaciones de ConexionesLaborales.....53-54
6.4.6. Relaciones de Conversaciones.....54
6.4.7. Relaciones de Reportes.....55
7. **BACKEND**.....56
7.1. Estructura del proyecto.....56

7.1.1. Estructura de directorios.....57
7.1.2. Descripción de directorios.....56
7.2. Persistencia de datos.....57
7.2.1. Conexión con la base de datos.....57
7.2.2. Mapeo objeto-relacional (ORM).....58
7.2.3. Gestión de sesiones.....59
7.2.4. Operaciones de persistencia.....60
7.3. Desarrollo de la API REST.....60
7.3.1. FastAPI.....60
7.3.2. Organización de los Endpoints.....61
7.3.3. Endpoints Principales.....61-65
7.3.4. Formato de intercambio de datos.....65
7.4. Autenticación y Autorización.....66
7.4.1. Registro e inicio de sesión.....66
7.4.2. Gestión de contraseñas.....67
7.4.3. Autenticación mediante JWT.....68
7.4.4. Control de acceso por roles.....69
7.5. Comunicación en Tiempo Real.....70
7.5.1. Implementación mediante WebSocket.....70
7.5.2. Flujo de comunicación.....71
7.6. Servicios Externos.....72
7.6.1. Cloudinary.....72
7.6.2. Brevo.....72
8. **FRONTEND**.....73
8.1. Arquitectura del Frontend.....73
8.1.1. Estructura del proyecto.....73
8.1.2. Descripción de directorios.....73-74
8.2. Comunicación con el Backend.....74
8.2.1. Capas de cliente.....75
8.2.2. Consumo de la API REST.....76
8.2.2.1. Patrón de servicios.....76
8.2.2.2. Endpoints principales.....77-82
8.2.2.3. Manejo de errores.....82
8.2.3. Gestión de autenticación.....83
8.2.3.1. Inicio de sesión.....83
8.2.3.2. Almacenamiento de la sesión.....83
8.2.3.3. Envío del token.....84
8.2.3.4. Registro de usuarios.....84
8.2.3.5. Recuperación de contraseña.....84
8.2.3.6. Expiración y cierre de sesión.....85

9. **IMPLEMENTACIÓN**……86
9.1. Desarrollo del software……86-87
9.2. Pruebas realizadas……87
9.2.1. Pruebas unitarias……87-88
9.2.2. Pruebas de integración……88-89
10. **CONCLUSIÓN**……90
10.1. Resultados obtenidos……90
10.2. Mejoras futuras……91
10.3. Reflexion final……91-92
11. **BIBLIOGRAFÍA**……93

1. INTRODUCCIÓN

1.1 Descripción del Proyecto

LaburAR es una plataforma digital diseñada para la intermediación laboral a nivel local. Su objetivo principal es conectar de forma directa y eficiente a empleadores y postulantes, centralizando la publicación y búsqueda de empleo en un entorno digital estructurado que facilite el encuentro entre la oferta y la demanda de trabajo en la comunidad.

1.2 Problemática y Justificación

Actualmente, la información sobre oportunidades laborales locales se encuentra dispersa en canales informales, redes sociales y publicaciones independientes. Esta fragmentación genera ineficiencia en el acceso a las ofertas, pérdida de tiempo para ambas partes y una rápida desactualización de las vacantes. La plataforma resuelve esta problemática al unificar la gestión de empleos en un solo sistema, optimizando los tiempos de búsqueda y mejorando la comunicación entre quienes ofrecen un puesto y quienes lo necesitan.

1.3 Alcance del sistema

El proyecto comprende el desarrollo de una solución enfocada en el ciclo completo de la intermediación laboral. Esto incluye la gestión de perfiles de usuarios diferenciados para postulantes y empleadores, un módulo para la publicación y visualización de ofertas con opciones de filtrado, y un sistema de postulación junto con un canal de comunicación interna para que las partes puedan coordinar de manera directa.

Como delimitación del sistema, la plataforma funcionará únicamente como un espacio de encuentro y vinculación entre postulantes y empleadores. Quedan fuera del alcance las integraciones con plataformas externas, los exámenes psicotécnicos automatizados y la gestión legal de contrataciones y liquidación de haberes.

2. ENTREVISTA

Objetivo: Conocer las necesidades y problemáticas existentes en los procesos de búsqueda y selección de personal, se realizaron entrevistas a diferentes comerciantes de establecimientos de la localidad. Se obtuvo información desde distintas perspectivas y detectar patrones comunes relacionados con la publicación de ofertas laborales, búsqueda de postulantes y recepción de solicitudes.

Entrevistas	Técnica de relevamiento	Fecha	Lugar	Entrevistador	Entrevistado	Modalidad	Duración
Entrevista Panaderia “La Tauriña” - Sede 1	Entrevista semiestructurada	20/7/2026	Florida 580	Nicolas Menin	Empleada 1	Presencial	10 min Aprox
Entrevista Panaderia “La Tauriña” -Sede 2	Entrevista semiestructurada	23/7/2026	Saavedra 39	Nicolas Menin	Empleada 1	Presencial	5 min Aprox
Entrevista Perfumería “Hiperfumerias”	Entrevista no estructurada	27/7/2026	Bernardo de Irigoyen 157	Nicolas Menin	Empleada 1	Presencial	10 min Aprox
Entrevista Tienda de ropa “Indumentaria Romina Escobar”	Entrevista no estructurada	3/8/2026	Murature 271	Nicolas Menin	Empleada 1	Presencial	15 min Aprox

2.1 Síntesis de los resultados

2.1.1 Entrevista 1 Panadería “La Tauriña” Sede 1

¿Cómo consiguió su empleo actual?

La persona entrevistada manifestó que consiguió su empleo principalmente a través del boca a boca y que posteriormente se acercó al establecimiento para presentar su currículum.

¿Ha utilizado alguna vez redes sociales o plataformas digitales para buscar oportunidades laborales? Si es así, ¿cuáles?

Indicó que ha utilizado redes sociales para buscar oportunidades laborales y como era de esperarse Facebook, aunque señaló que estas búsquedas generalmente no llegan a concretarse en una oportunidad laboral.

¿Cómo realizan actualmente la búsqueda de personal cuando necesitan cubrir un puesto?

Según lo expresado, la búsqueda de personal se realiza principalmente mediante conocidos y recomendaciones personales, es decir, mediante el boca a boca. También mencionó el uso de carteles colocados en el exterior del establecimiento para comunicar la búsqueda.

Desde su experiencia, ¿qué dificultades ha encontrado al momento de buscar empleo o encontrar personas para cubrir un puesto?

Entre las dificultades mencionadas se destacó la posibilidad de que una persona no llegue a ser considerada para una oportunidad laboral, así como la influencia que pueden tener los contactos personales y familiares al momento de acceder a determinados puestos.

¿Considera que sería útil contar con una plataforma que facilite el contacto entre personas que buscan empleo y establecimientos que necesitan incorporar personal? ¿Por qué?

Consideró que sería una buena alternativa, especialmente si la plataforma permite realizar el proceso de manera sencilla y ordenada, evitando la saturación de publicaciones y mensajes que pueden producirse mediante otros medios utilizados actualmente.

2.1.2 Entrevista 2 Panadería “La Tauriña” Sede 2

¿Cómo consiguió su empleo actual?

Comentó que consiguió su trabajo al acercarse personalmente al establecimiento para dejar su currículum.

¿Ha utilizado alguna vez redes sociales o plataformas digitales para buscar oportunidades laborales? Si es así, ¿cuáles?

Sí, utilizó redes sociales como Facebook para buscar trabajo. Comentó que normalmente no llegan a nada pero tuvo un conocido que si llego a tener un trabajo por una publicación.

¿Cómo realizan actualmente la búsqueda de personal cuando necesitan cubrir un puesto?

Comentó que normalmente buscan personas mediante conocidos y recomendaciones. También utilizan carteles colocados afuera del negocio como en la anterior entrevista.

Desde su experiencia, ¿qué dificultades ha encontrado al momento de buscar empleo o encontrar personas para cubrir un puesto?

Mencionó que muchas veces para acceder a una oportunidad laboral es importante contar con conocidos o contactos que puedan recomendar a la persona.

¿Considera que sería útil contar con una plataforma que facilite el contacto entre personas que buscan empleo y establecimientos que necesitan incorporar personal? ¿Por qué?

La idea le pareció buena y mostró interés cuando le expliqué que estaba desarrollando un sistema de intermediación laboral a nivel local. A partir de la explicación sobre cómo funcionaría, propuso que el postulante pueda tener su currículum disponible dentro de la plataforma, para aprovechar esa información al momento de completar su perfil y hacerlo de una manera más rápida y sencilla.

2.1.3 Entrevista 3 Perfumeria “Hiperfumerias”

Síntesis de la conversación:

Durante la charla se le comentó a la persona entrevistada la idea general del sistema de intermediación laboral a nivel local. La propuesta le pareció interesante y comentó que, en su caso, suele enviar el currículum mediante un número de teléfono cuando busca trabajo.

También explicó que parte de las oportunidades laborales se gestionan desde Bahía Blanca, debido a que la empresa cuenta con otra sede en esa ciudad. Por otro lado, coincidió en que es poco frecuente encontrar trabajo mediante redes sociales y consideró que sería útil contar con una página enfocada exclusivamente en Punta Alta, donde se puedan concentrar las ofertas y búsquedas laborales de la ciudad.

2.1.4 Entrevista — Tienda de ropa “ Romina Escobar”

Síntesis de la conversación:

Durante la charla se comentó la idea general del sistema de intermediación laboral a nivel local. La persona entrevistada estuvo de acuerdo en que actualmente puede resultar difícil encontrar trabajo mediante redes sociales, aunque aclaró que no es imposible, ya que ella misma consiguió su trabajo actual a través de este medio.

También comentó cómo dan a conocer las oportunidades laborales en el establecimiento. Actualmente utilizan principalmente Instagram, mediante

publicaciones e historias. A partir de esto, se identificó una dificultad relacionada con el alcance de este medio, ya que las publicaciones e historias dependen en gran medida de las personas que siguen la cuenta, por lo que una oferta laboral puede no llegar a personas que podrían estar interesadas pero que no conocen o no siguen el perfil del establecimiento.

2.2 Necesidades y problemáticas identificadas

A partir de las entrevistas realizadas se pudieron identificar distintas dificultades relacionadas con la búsqueda y publicación de oportunidades laborales en el ámbito local. Si bien cada persona entrevistada tuvo experiencias diferentes, se encontraron algunos puntos en común.

Una de las principales dificultades mencionadas fue la dependencia de contactos personales y recomendaciones para conseguir empleo. En algunas de las entrevistas se comentó que el acceso a determinadas oportunidades laborales se produce mediante conocidos, familiares o el boca a boca, lo que puede dificultar que otras personas puedan conocer y acceder a dichas oportunidades.

También se observó que las redes sociales son utilizadas tanto para buscar empleo como para publicar ofertas laborales, aunque su efectividad presenta ciertas limitaciones. Algunas de las personas entrevistadas comentaron que han buscado trabajo mediante redes sociales sin obtener resultados, mientras que otras señalaron que sí es posible conseguir empleo por estos medios. En este sentido, las redes sociales no fueron consideradas como un medio inútil, sino como una alternativa que puede funcionar, pero que no está específicamente orientada a la búsqueda laboral.

Desde el punto de vista de los empleadores, se identificó además una dificultad relacionada con el alcance de las publicaciones. Por ejemplo, la utilización de Instagram para comunicar una búsqueda laboral depende en gran medida de las personas que siguen la cuenta del establecimiento, por lo que la oferta puede no llegar a otros posibles postulantes.

Otro aspecto identificado fue la falta de un espacio centralizado para las oportunidades laborales de la localidad. Durante una de las entrevistas surgió la idea de contar con una página dedicada específicamente a Punta Alta, donde puedan concentrarse las ofertas laborales y las personas interesadas en encontrar empleo. Esto permitiría evitar que la información se encuentre distribuida entre redes sociales, contactos personales, carteles y otros medios.

Por último, se identificó la necesidad de facilitar la presentación de la información laboral de los postulantes. Durante una de las entrevistas se propuso que el currículum pueda encontrarse disponible dentro del perfil del postulante, permitiendo aprovechar esa información al momento de completar su perfil y agilizando el proceso.

3. REQUERIMIENTOS

3.1 Requerimientos funcionales

ID	Requerimiento	Actor	Caso de Uso
RF-01	Registro de postulante	Postulante	DCU-Postulante
RF-02	Registro de empleador	Empleador	DCU-Empleador
RF-03	Inicio de sesión	Usuario	DCU-General
RF-04	Completar perfil profesional	Postulante	DCU-Postulante
RF-05	Modificar perfil profesional	Postulante	DCU-Postulante
RF-06	Cargar currículum	Postulante	DCU-Postulante
RF-07	Buscar ofertas laborales	Postulante	DCU-Postulante
RF-08	Filtrar ofertas laborales	Postulante	DCU-Postulante
RF-09	Publicar oferta laboral	Empleador	DCU-Empleador
RF-10	Buscar postulantes	Empleador	DCU-Empleador
RF-11	Filtrar postulantes	Empleador	DCU-Empleador
RF-12	Postularse a ofertas laborales	Postulante	DCU-Postulante
RF-13	Gestionar postulaciones	Empleador	DCU-Empleador
RF-14	Aceptar postulaciones	Empleador	DCU-Empleador
RF-15	Aceptar solicitudes laborales	Postulante	DCU-Postulante
RF-16	Mensajería entre usuarios	Postulante / Empleador	DCU-General
RF-17	Realizar reportes	Usuario	DCU-General

RF-18	Recuperar contraseña	Usuario	DCU-General
RF-19	Cerrar sesión	Usuario	DCU-General
RF-20	Panel de control	Administrador	DCU-Administrador
RF-21	Visualizar usuarios	Administrador	DCU-Administrador
RF-22	Gestión de usuarios	Administrador	DCU-Administrador
RF-23	Suspender usuarios	Administrador	DCU-Administrador
RF-24	Visualizar reportes	Administrador	DCU-Administrador
RF-25	Gestionar Reportes	Administrador	DCU-Administrador
RF-26	Atender Reporte	Administrador	DCU-Administrador

3.2 Requerimientos no funcionales

ID	Requerimiento	Descripción
RNF-01	Autenticación segura de usuarios	El sistema deberá garantizar un acceso seguro mediante autenticación de usuarios y validación de credenciales.
RNF-02	Interfaz intuitiva	La interfaz deberá proporcionar una experiencia de uso clara, organizada y fácil de comprender para los usuarios.
RNF-03	Compatibilidad con navegadores web	El sistema deberá ser compatible con los principales navegadores web, incluyendo Google Chrome, Mozilla Firefox, Microsoft Edge y Brave.
RNF-04	Tiempo de respuesta eficiente	El sistema no deberá superar los 10 segundos de respuesta durante la carga de interfaces y operaciones principales.
RNF-05	Disponibilidad del sistema	El sistema deberá mantenerse disponible para los usuarios durante su funcionamiento, garantizando estabilidad y acceso continuo a la plataforma.

RNF-06	Integridad de la información	El sistema deberá garantizar la consistencia y correcto almacenamiento de la información registrada por los usuarios.
RNF-07	Escalabilidad de la aplicación	La aplicación deberá permitir futuras ampliaciones y mejoras sin afectar significativamente la estructura principal del sistema.
RNF-08	Validación de formularios	El sistema deberá validar los datos ingresados en formularios para evitar errores y registros inválidos.
RNF-09	Navegación accesible	La navegación dentro de la plataforma deberá mantenerse organizada y accesible para facilitar el uso de las distintas funcionalidades.
RNF-10	Protección de contraseñas	Las contraseñas almacenadas en el sistema deberán encontrarse cifradas para proteger la información de los usuarios.
RNF-11	Feedback de espera	El sistema deberá informar al usuario cuando una operación se encuentre en proceso de carga o espera.

4. ANÁLISIS DEL SISTEMA

4.1 Perfiles de usuarios

4.1.1 Postulante

Usuario orientado a la búsqueda activa de empleo. Sus capacidades principales incluyen el acceso a la cartelera de vacantes, la gestión de un perfil profesional dinámico y la postulación directa a oportunidades de interés.

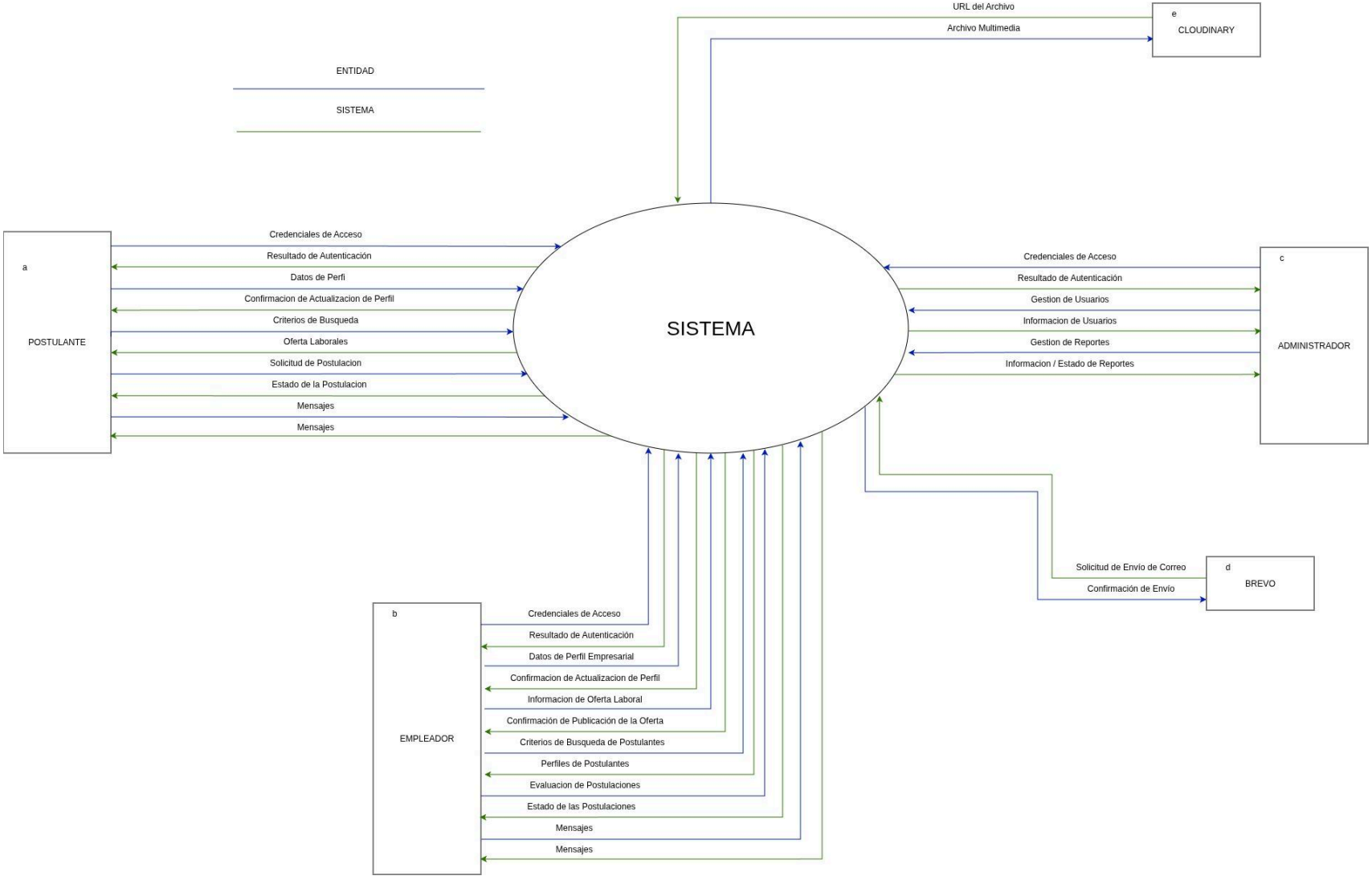
4.1.2 Empleador

Usuario orientado a la gestión de recursos humanos y talento. Sus capacidades se centran en la publicación y administración de ofertas laborales, así como en la visualización y contacto con los perfiles postulantes que mejor se ajusten al requerimiento técnico de la vacante.

4.1.3 Administrador

Usuario con privilegios de supervisión y control operativo sobre el ecosistema. Su rol es crítico para garantizar la integridad, seguridad y correcto funcionamiento de la plataforma. Sus responsabilidades principales incluyen la gestión del ciclo de vida de los usuarios (altas, bajas y moderación), la supervisión de las operaciones del sistema y la extracción de métricas o reportes de rendimiento que permiten evaluar el estado de la plataforma.

4.2 Diagrama Contextual



4.3 Tabla de eventos

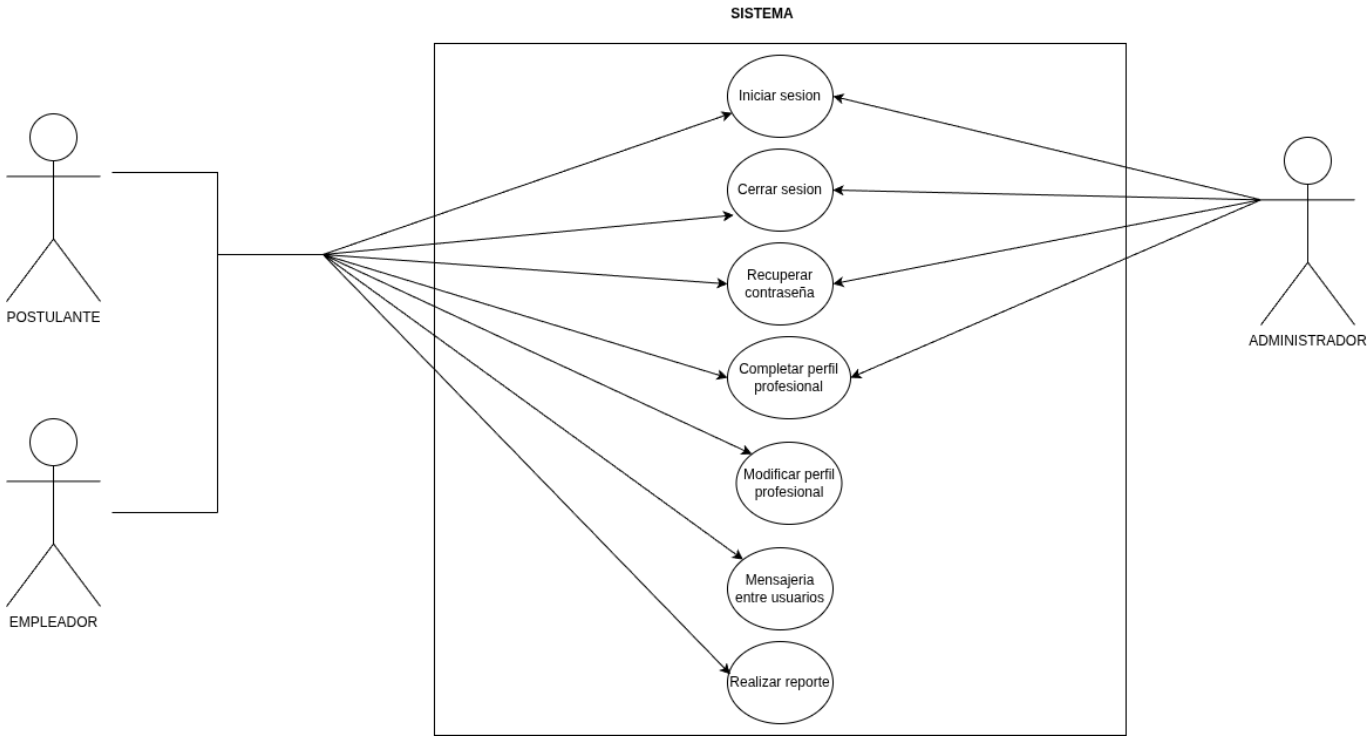
Tipo	Entidad Externa	Descripción	Estímulo	Respuesta	Función Asociada
Externo	Usuario	El postulante inicia sesión en la plataforma.	Credenciales de Acceso	Resultado de Autenticación	Autenticar usuario
Externo	Postulante	El postulante actualiza la información de su perfil profesional.	Datos de Perfil	Confirmación de Actualización de Perfil	Gestionar perfil
Externo	Postulante	El postulante realiza una búsqueda de ofertas laborales.	Criterios de Búsqueda	Ofertas Laborales	Buscar ofertas laborales
Externo	Postulante	El postulante solicita postularse a una oferta laboral.	Solicitud de Postulación	Estado de la Postulación	Gestionar postulaciones
Externo	Postulante	El postulante intercambia mensajes con otros usuarios.	Mensajes	Mensajes	Gestionar mensajería
Externo	Empleador	El empleador actualiza la información de su perfil empresarial.	Datos de Perfil Empresarial	Confirmación de Actualización de Perfil	Gestionar perfil empresarial
Externo	Empleador	El empleador publica una oferta laboral.	Información de Oferta Laboral	Confirmación de Publicación de la Oferta	Gestionar ofertas laborales
Externo	Empleador	El empleador realiza una búsqueda de postulantes.	Criterios de Búsqueda de Postulantes	Perfiles de Postulantes	Buscar postulantes

Externo	Empleador	El empleador evalúa las postulaciones recibidas.	Evaluación de Postulaciones	Estado de las Postulaciones	Gestionar postulaciones
Externo	Empleador	El empleador intercambia mensajes con los postulantes.	Mensajes	Mensajes	Gestionar mensajería
Externo	Administrador	El administrador gestiona la información de los usuarios registrados en la plataforma.	Gestión de Usuarios	Información de Usuarios	Gestionar usuarios
Externo	Administrador	El administrador revisa y administra los reportes generados por los usuarios.	Gestión de Reportes	Información / Estado de Reportes	Gestionar reportes
Servicio Externo	Brevo	El sistema solicita el envío de un correo electrónico mediante el servicio de Brevo.	Solicitud de Envío de Correo	Confirmación de Envío	Enviar correo electrónico
Servicio Externo	Cloudinary	El sistema almacena archivos multimedia en Cloudinary para su posterior acceso.	Archivo Multimedia	URL del Archivo	Almacenar archivos multimedia

4.4 Casos de uso

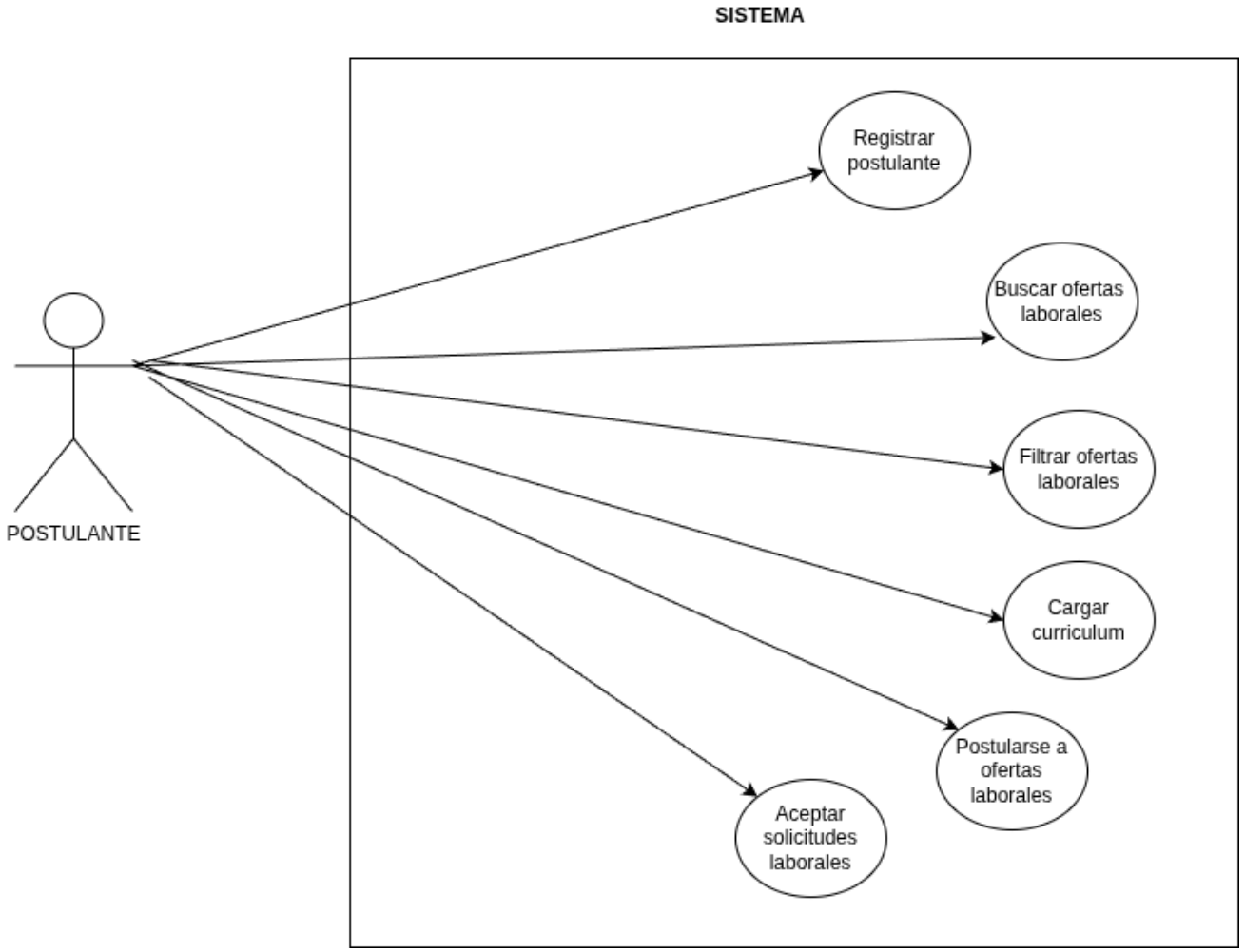
4.4.1 DCU-General

Elementos	Descripción
Nombre	DCU-General
Actores	Postulante / Empleador/ Administrador
Descripción	Representa las funcionalidades generales del sistema relacionadas con el acceso y cierre de sesión de los usuarios. Además, contempla la comunicación entre postulantes y empleadores dentro de la plataforma.



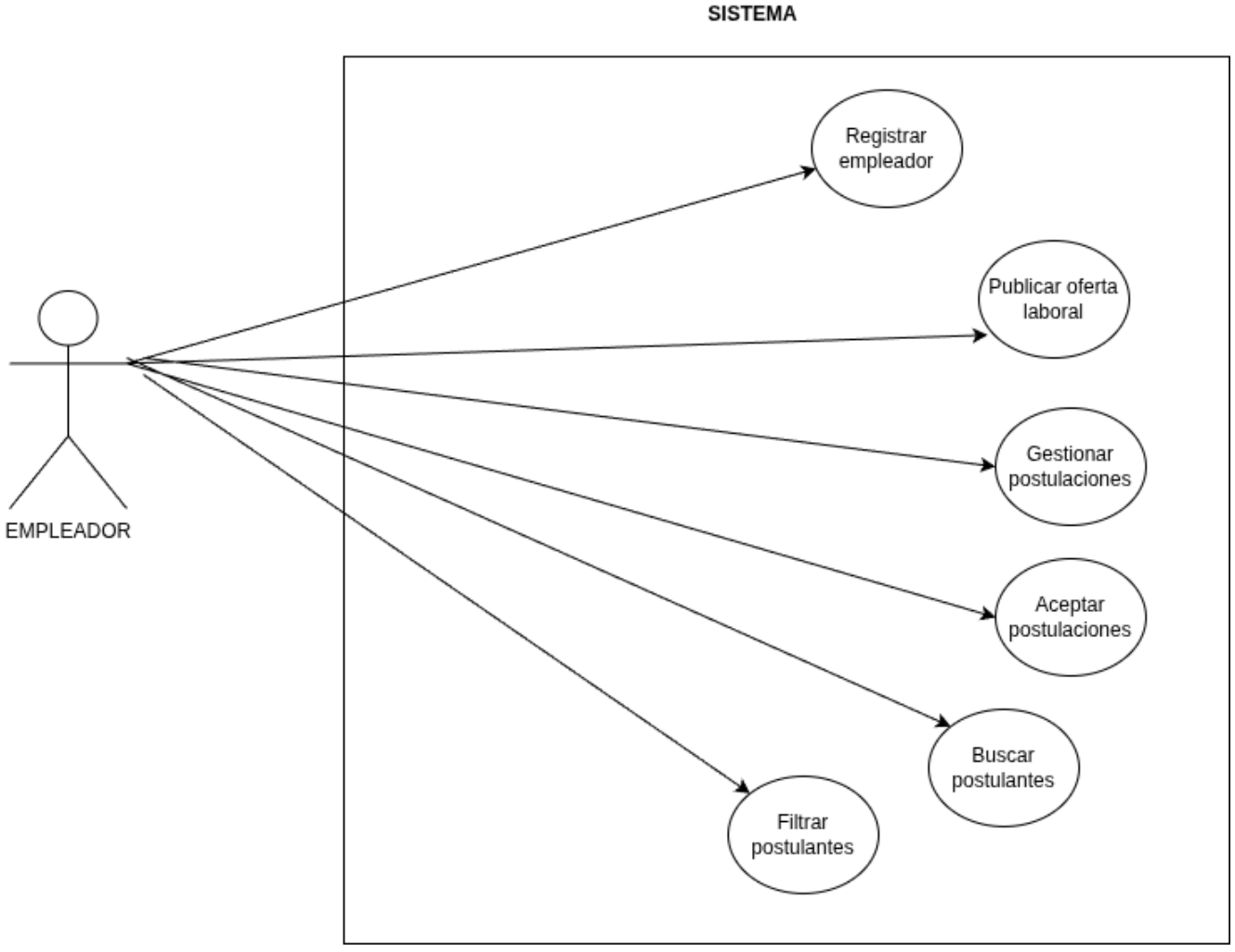
4.4.2 DCU-Postulante

Elementos	Descripción
Nombre	DCU-Postulante
Actores	Postulante
Descripción	Representa las funcionalidades disponibles para el usuario postulante dentro de la plataforma, relacionadas con la búsqueda y postulación a ofertas laborales.



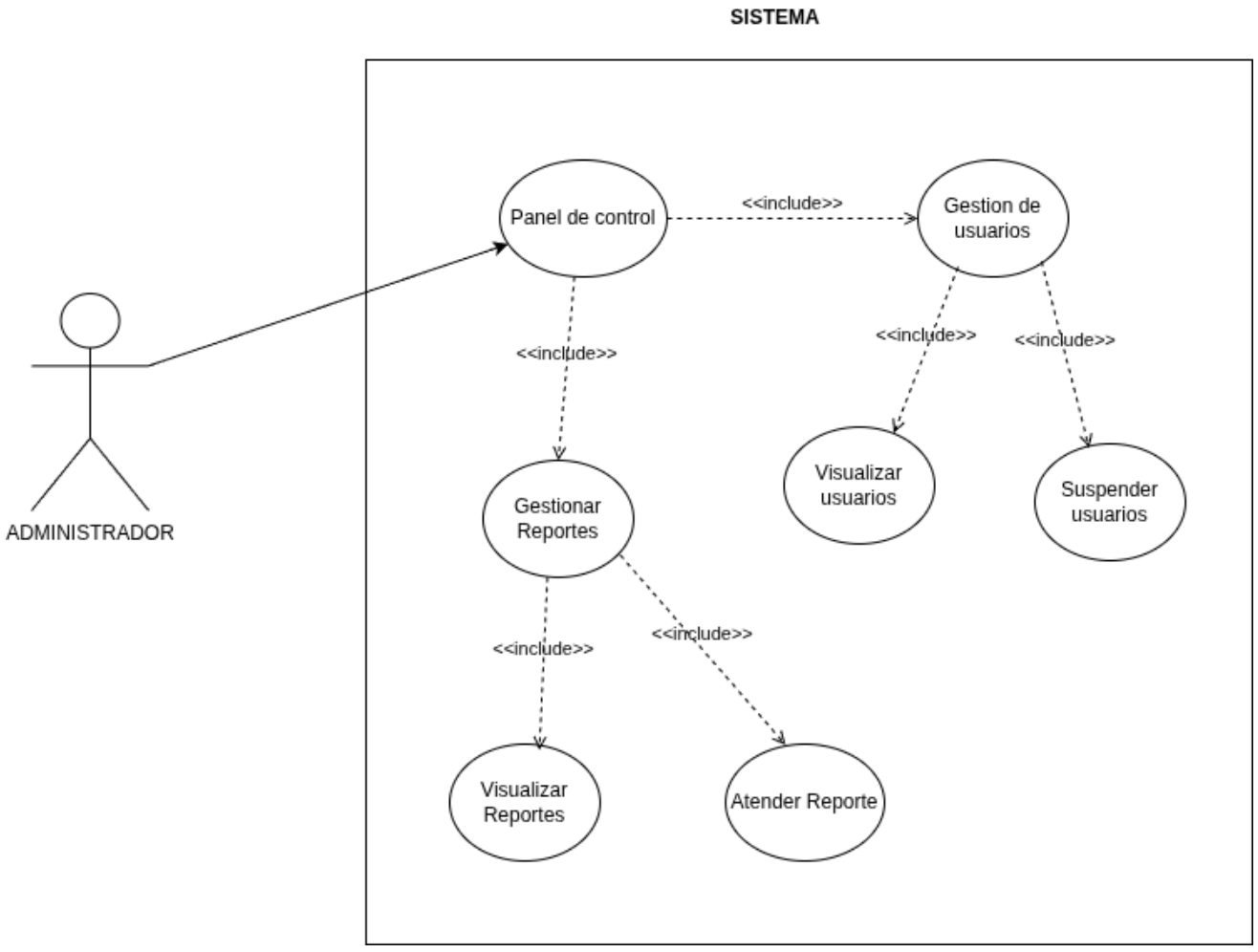
4.4.3 DCU-Empleador

Elementos	Descripción
Nombre	DCU-Empleador
Actores	Empleador
Descripción	Representa las funcionalidades disponibles para el usuario empleador dentro de la plataforma, relacionadas con la publicación de ofertas laborales y la gestión de postulaciones.



4.4.4 DCU-Administrador

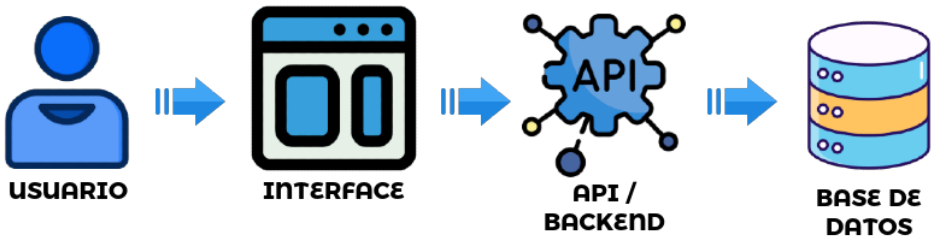
Elementos	Descripción
Nombre	DCU-Administrador
Actores	Administrador
Descripción	Representa las funcionalidades disponibles para el administrador dentro de la plataforma, relacionadas con la supervisión del sistema y la gestión de usuarios.



5. DISEÑO DEL SISTEMA

5.1 Arquitectura del sistema

El sistema adopta una arquitectura de tipo cliente-servidor, diseñada para garantizar el desacoplamiento entre la capa de presentación y la lógica de procesamiento. La comunicación se establece mediante una API REST, que actúa como interfaz estandarizada para el intercambio de datos entre el frontend y el backend



Para asegurar la robustez y la escalabilidad, la implementación interna sigue un patrón de diseño en capas, estructurado de la siguiente manera:

- Capa de Presentación: Responsable de la interfaz de usuario y la captura de eventos, garantizando una interacción fluida.
- Capa de Lógica de Negocio: Encargada de procesar las reglas del sistema, validaciones y orquestación de servicios.
- Capa de Acceso a Datos: Gestiona la persistencia y recuperación de información, aislando la lógica de la base de datos de los componentes superiores.

También, el sistema integra WebSockets. Esta tecnología permite la comunicación bidireccional en tiempo real, optimizando el rendimiento al evitar el sondeo constante del cliente hacia el servidor, siendo esencial para el módulo de mensajería.

5.2 Diseño visual

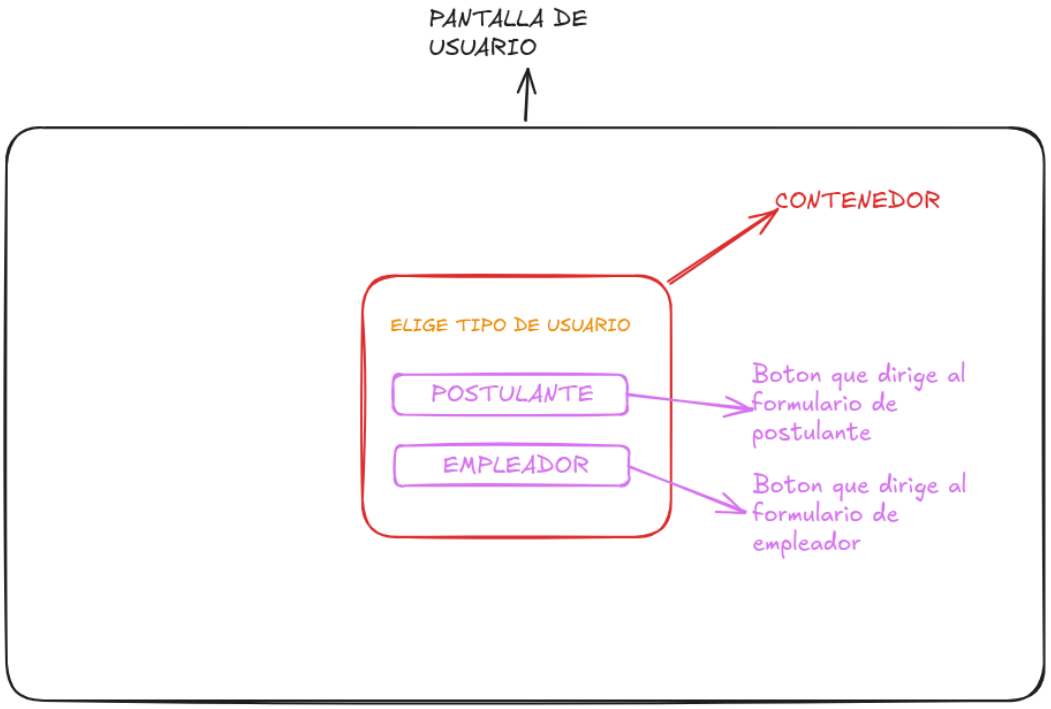
El diseño visual se fundamenta en los principios de diseño minimalista y centrado en el usuario. El objetivo principal es reducir la carga cognitiva mediante una jerarquía visual clara, permitiendo que tanto el postulante como el empleador localicen la información crítica de manera eficiente y sin distracciones ornamentales.

Los criterios rectores aplicados durante el desarrollo de las interfaces son:

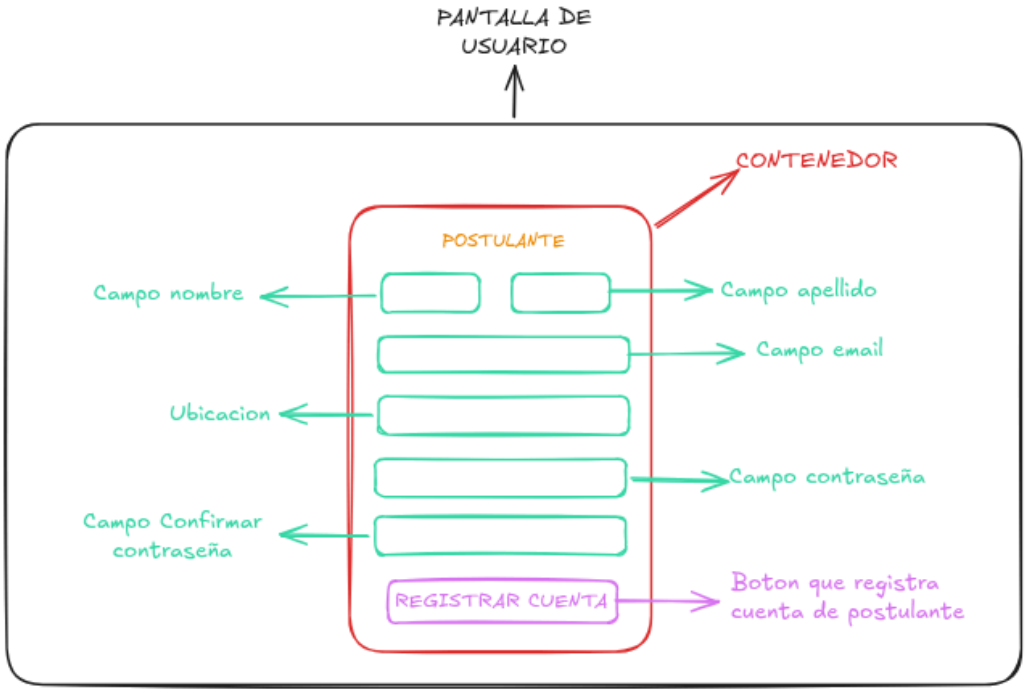
- Consistencia Visual: Se ha mantenido una identidad gráfica unificada a lo largo de todos los módulos, utilizando una paleta de colores y componentes tipográficos coherentes que facilitan el reconocimiento del sistema y refuerzan la confianza del usuario.
- Jerarquía de Información: La disposición de los elementos prioriza la visibilidad de los datos esenciales, utilizando un enfoque de diseño modular que permite un escaneo visual rápido.
- Accesibilidad y Usabilidad: La estructura de navegación se ha simplificado para garantizar un flujo intuitivo. Los controles de acción principal fueron diseñados para asegurar su accesibilidad, minimizando los clics necesarios para completar procesos críticos, como la postulación a un empleo o la publicación de una vacante.

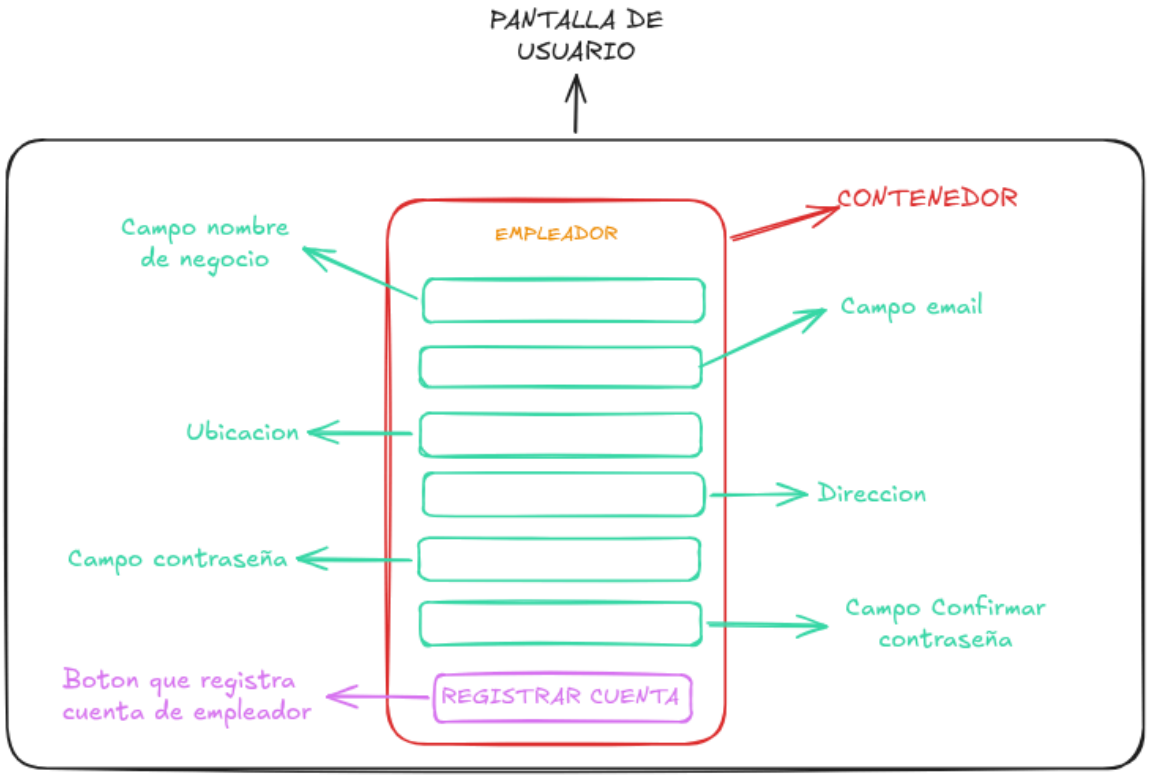
5.3 Interfaz de Acceso y Generales

5.3.1 Interfaz de selección de usuario

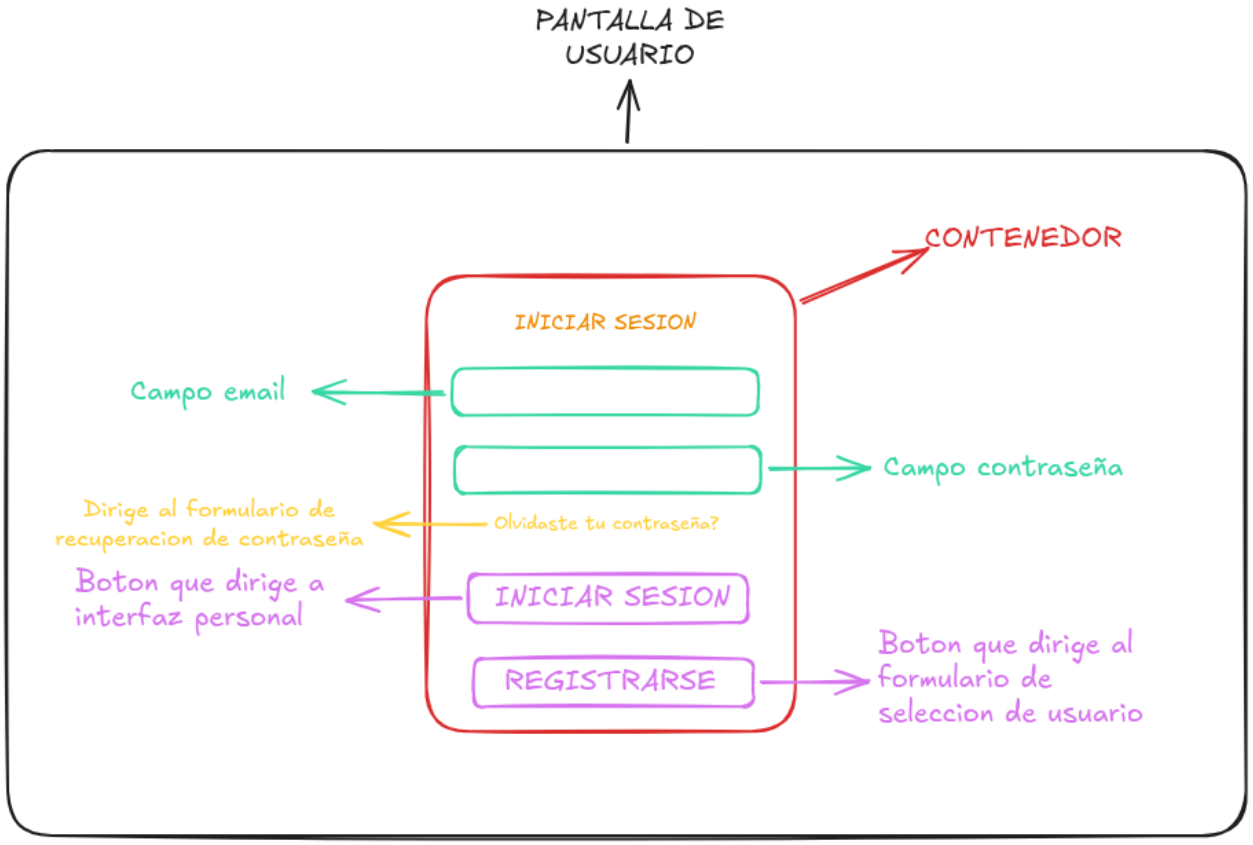


5.3.2 Interfaz de registro

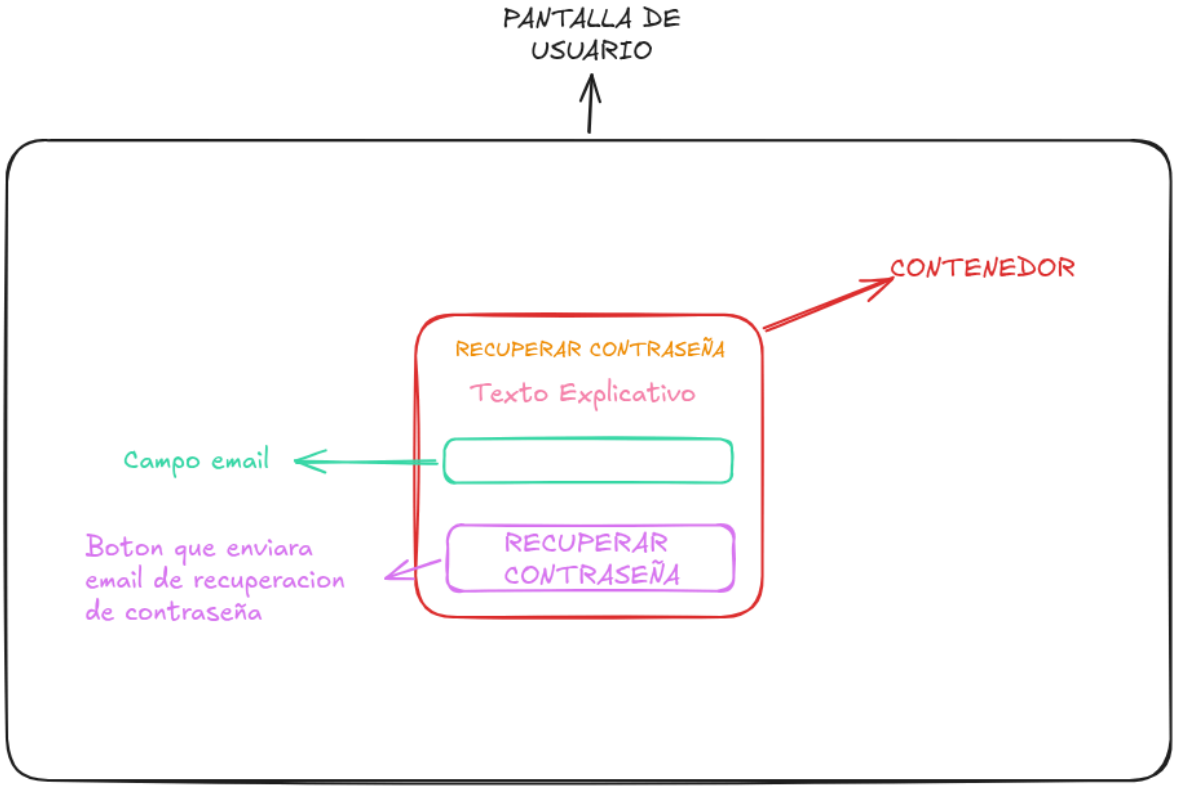


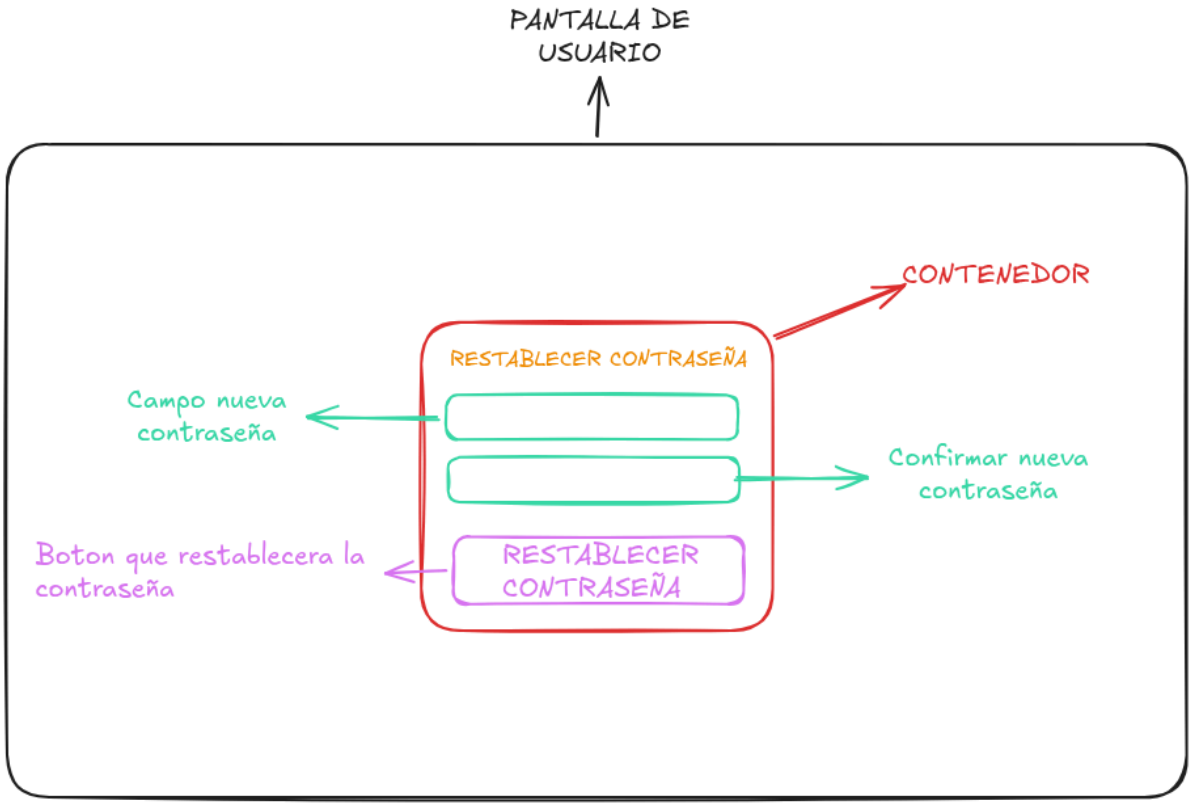


5.3.3 Interfaz de inicio de sesión



5.3.4 Interfaz de recuperación de contraseña

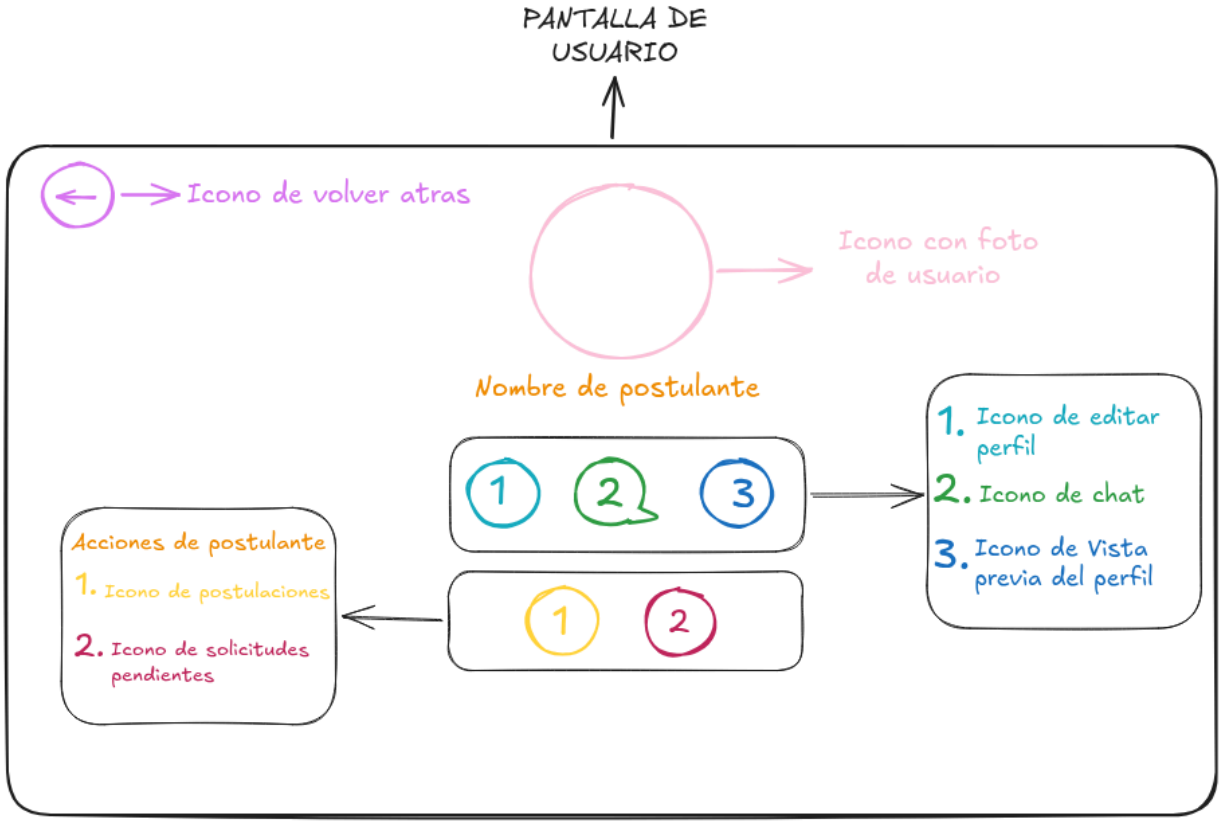




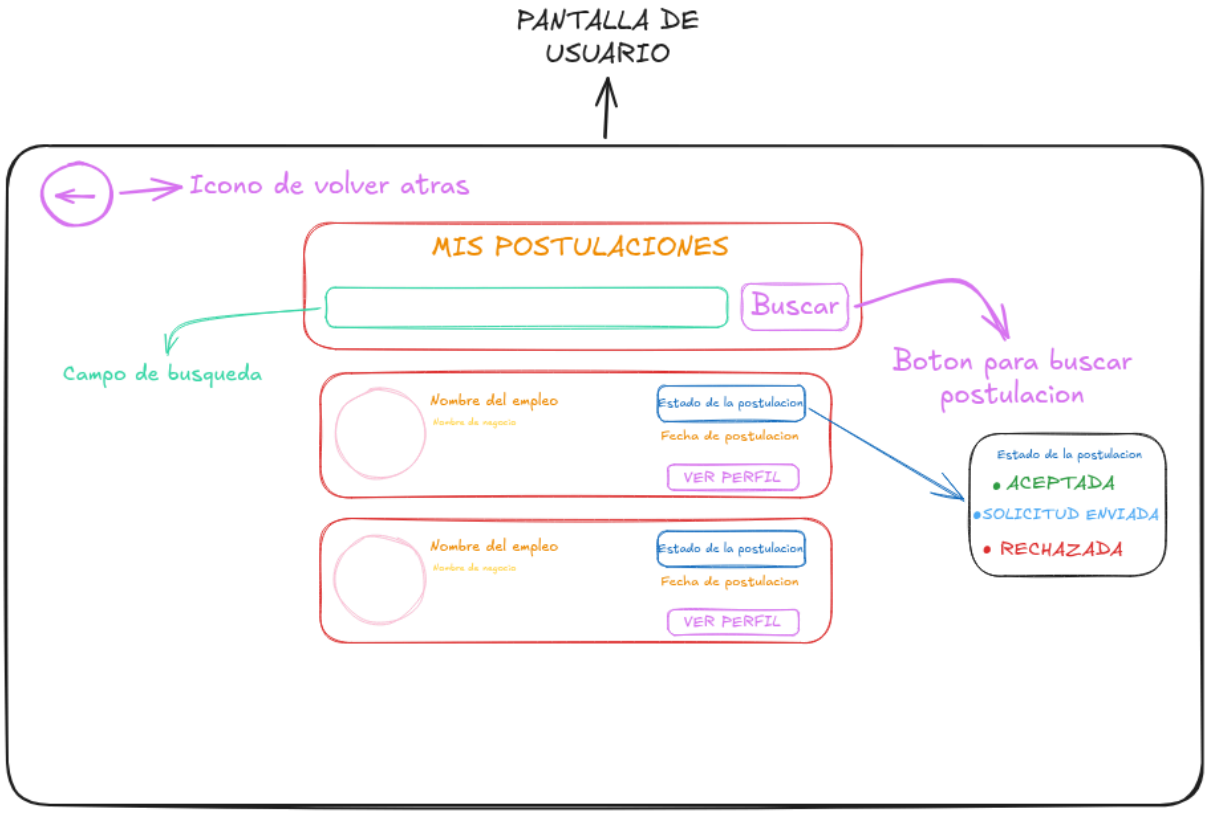
5.4 Módulo de postulante

La interfaz de postulante representa el entorno destinado a los usuarios que buscan oportunidades laborales dentro de la plataforma. Esta interfaz permite administrar la información personal del usuario, visualizar ofertas de empleo, gestionar postulaciones realizadas y mantener comunicación con empleadores mediante el sistema de mensajería integrado.

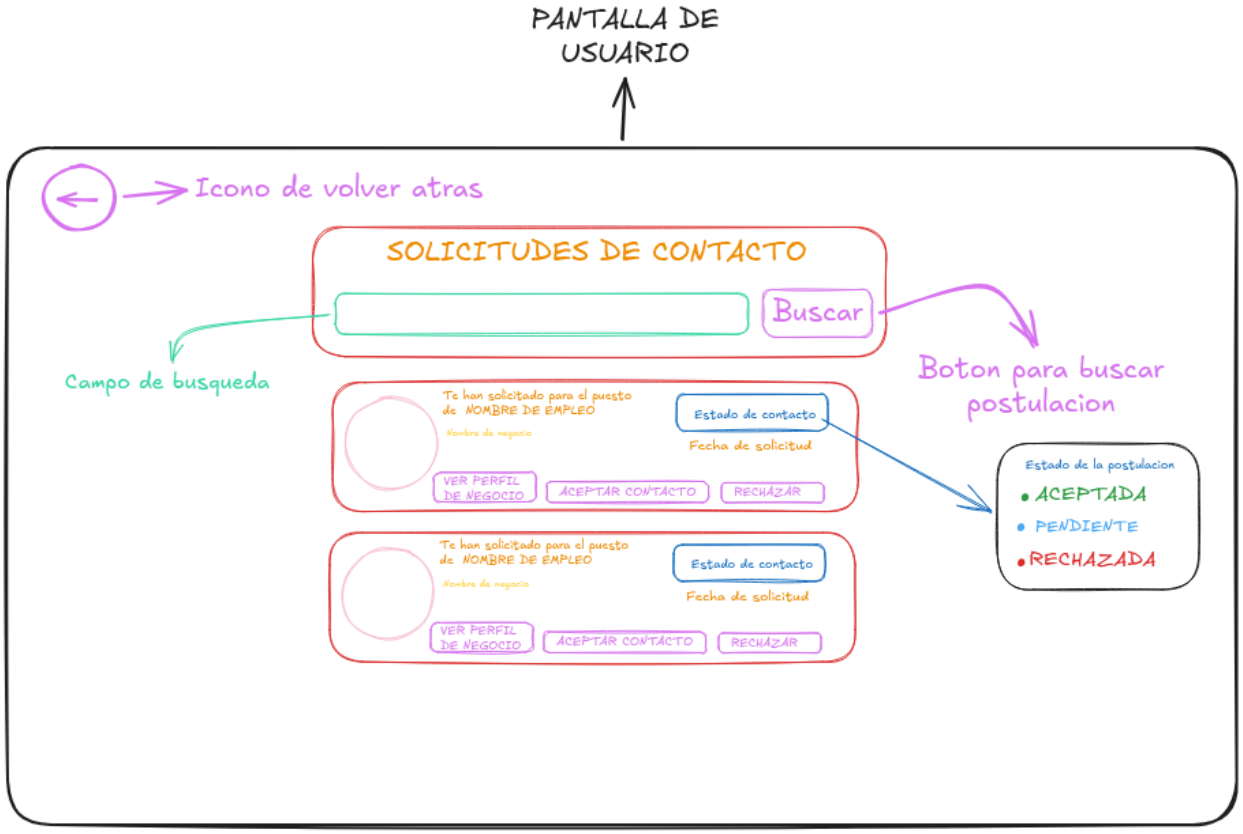
5.4.1 Perfil personal



5.4.2 Interfaz de postulaciones



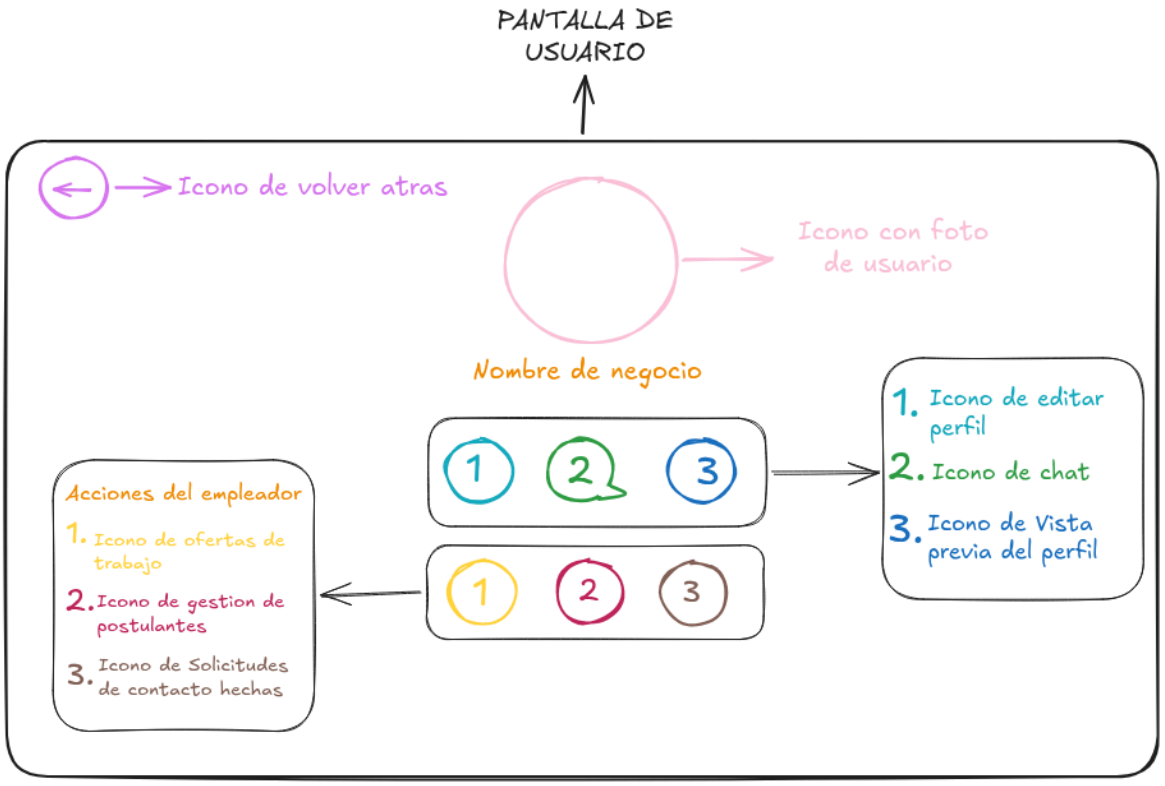
5.4.3 Solicitudes de contacto



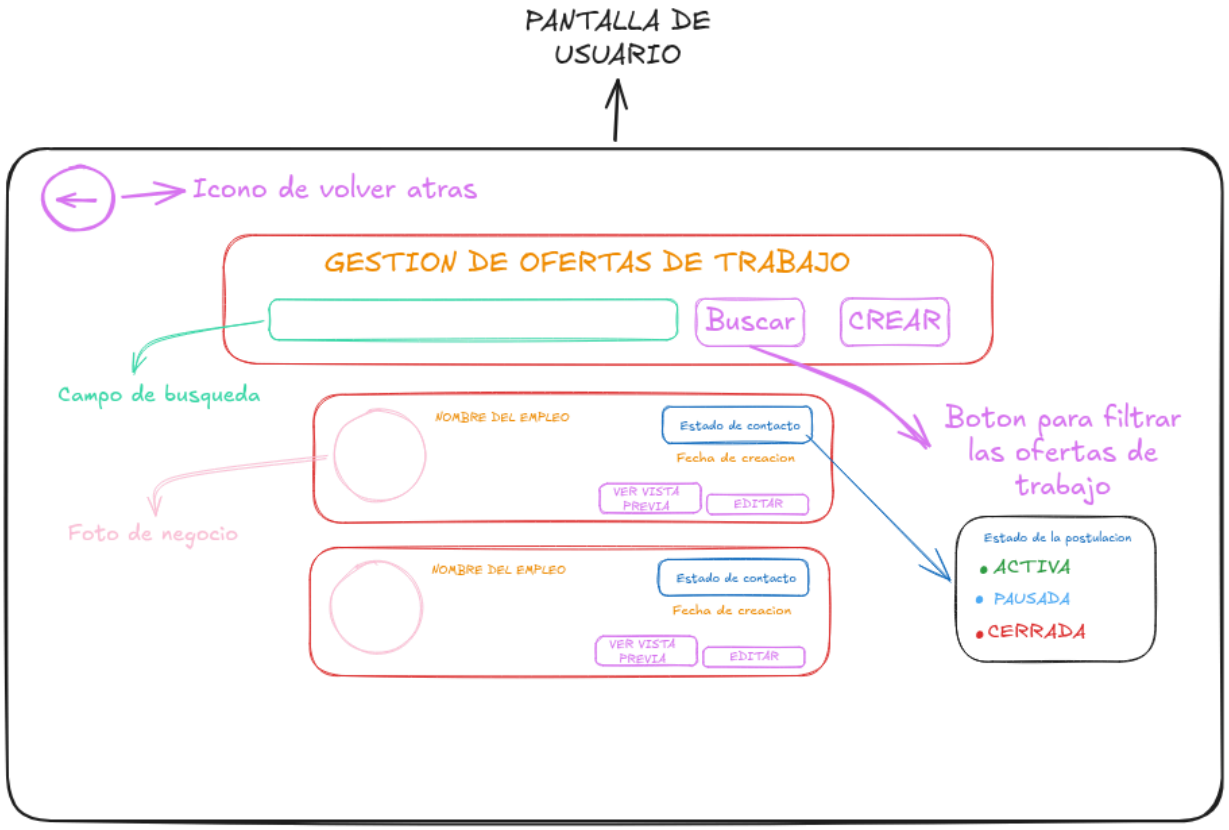
5.5 Módulo de empleador

La sección correspondiente al empleador reúne las interfaces y funcionalidades destinadas a negocios dentro de la plataforma. Mediante estas vistas, el usuario empleador puede administrar información del negocio, publicar ofertas laborales, visualizar postulantes y gestionar solicitudes de contacto y comunicación con posibles candidatos.

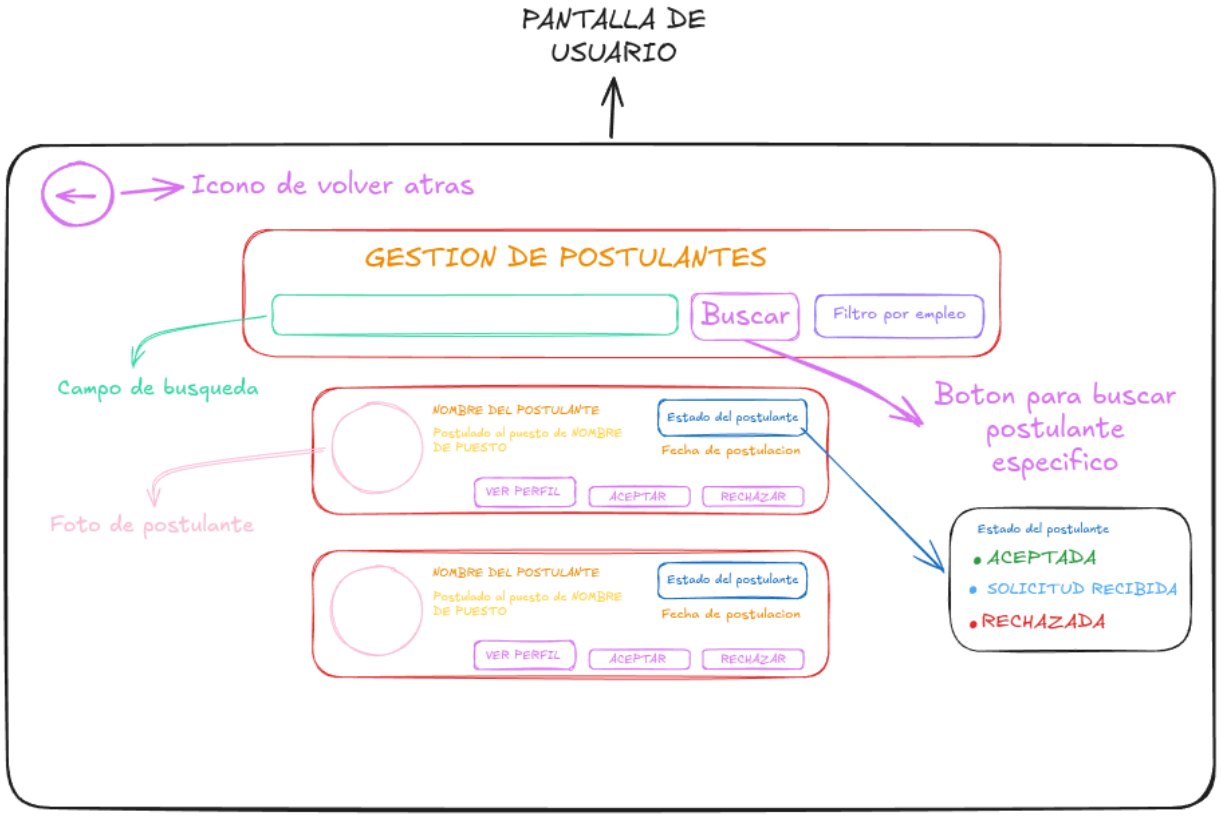
5.5.1 Perfil personal



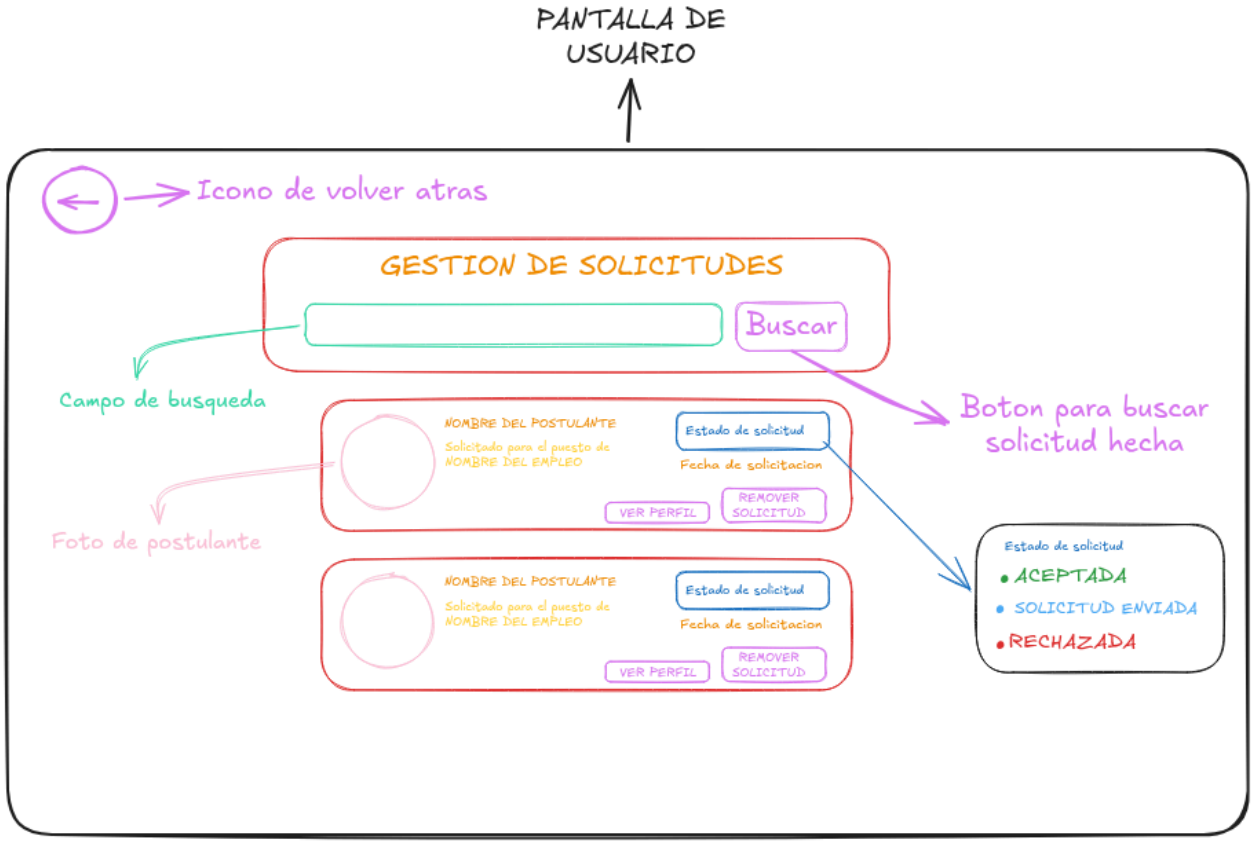
5.5.2 Gestión de ofertas



5.5.3 Gestión de postulantes



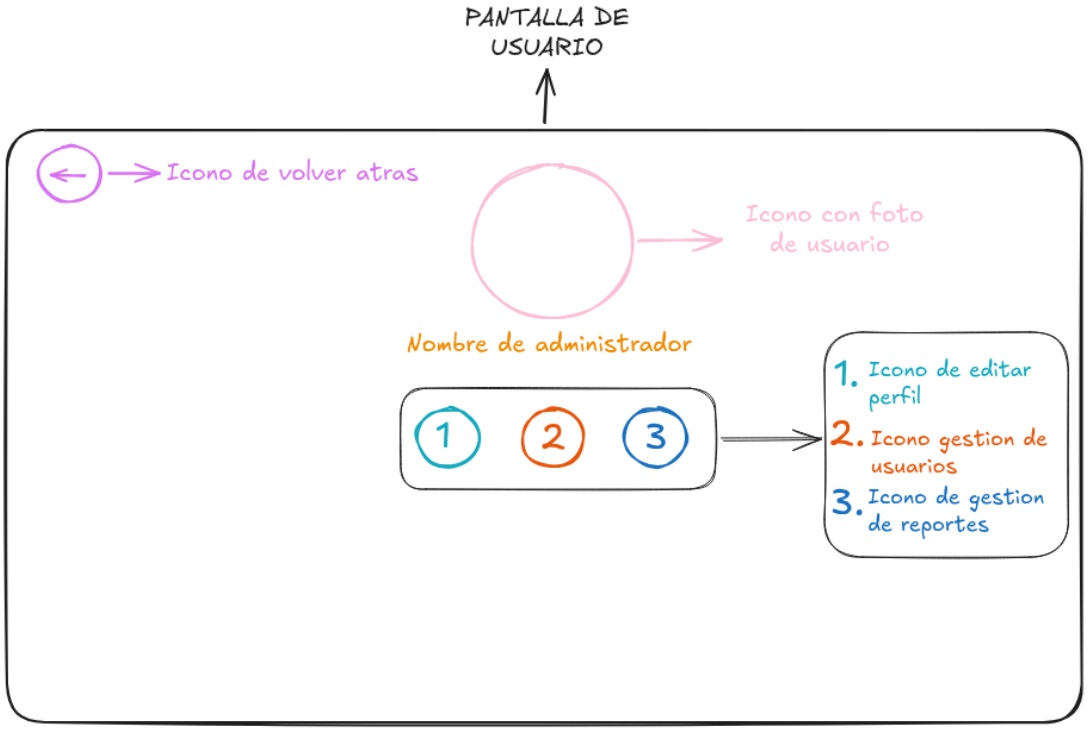
5.5.4 Gestión de solicitud es de contacto



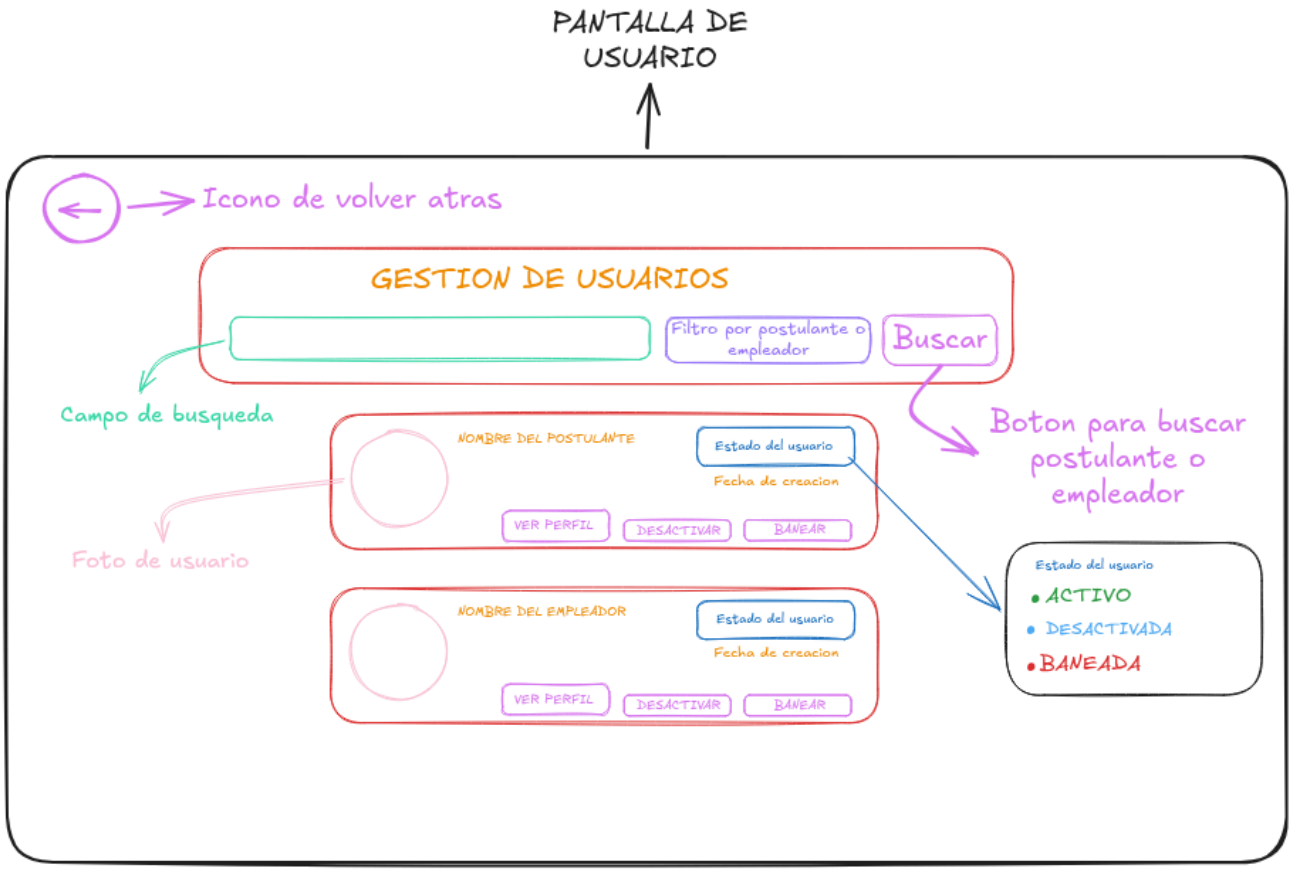
5.6 Módulo de administración

La sección correspondiente al administrador reúne las interfaces y funcionalidades orientadas a la supervisión y control general de la plataforma. Estas vistas permiten gestionar usuarios, visualizar reportes y administrar distintas herramientas relacionadas con el funcionamiento y moderación del sistema.

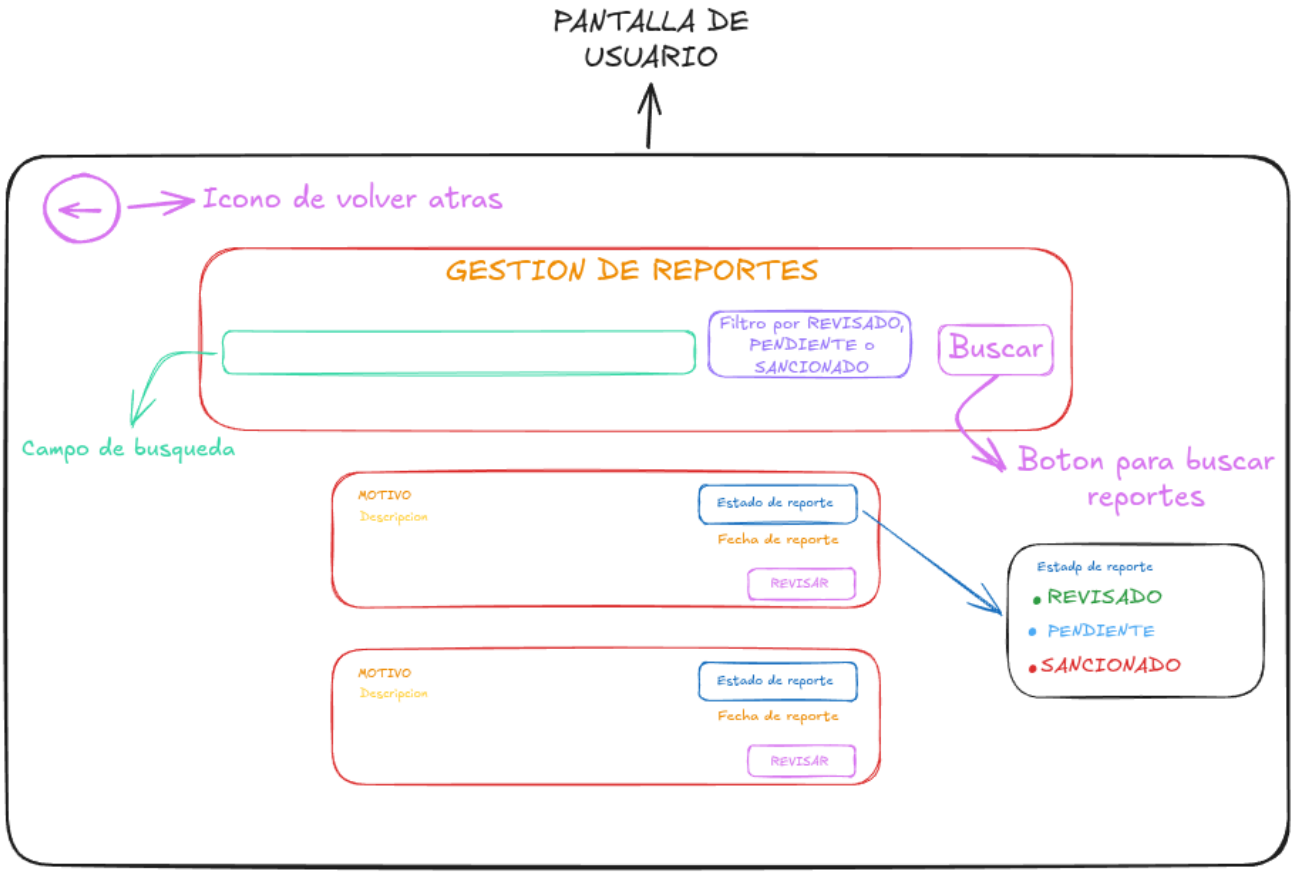
5.6.1 Perfil personal



5.6.2 Gestión de usuarios

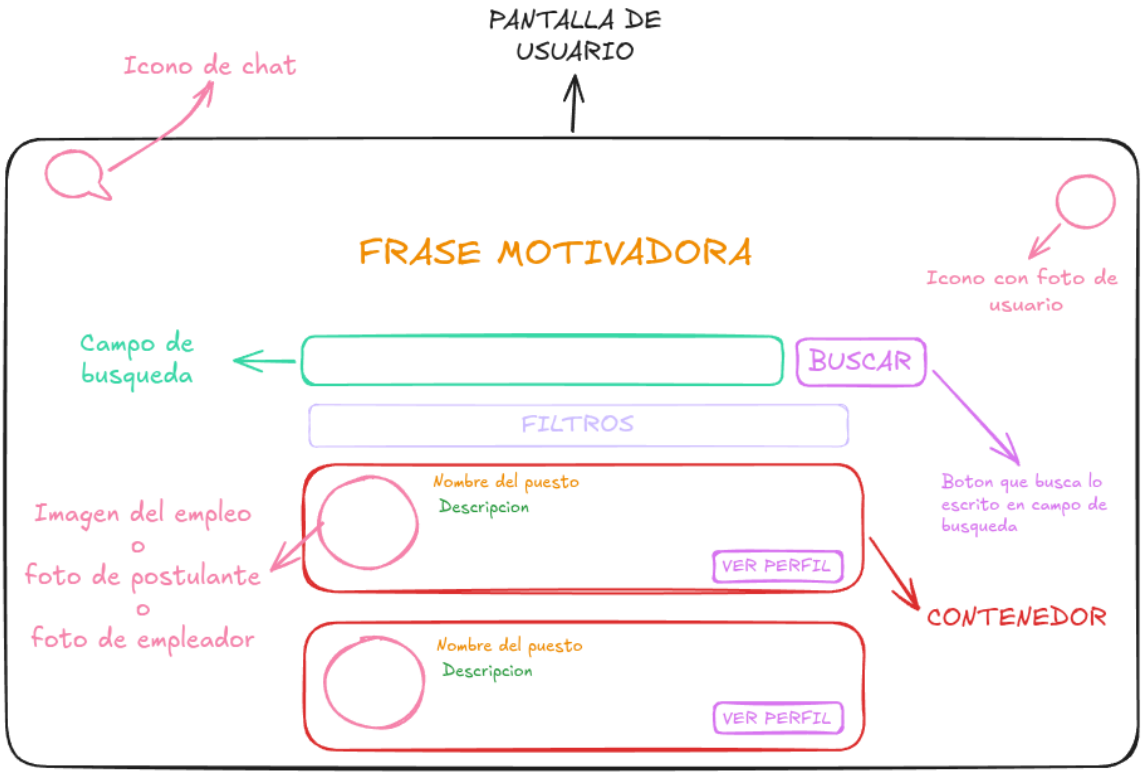


5.6.3 Gestión de reportes

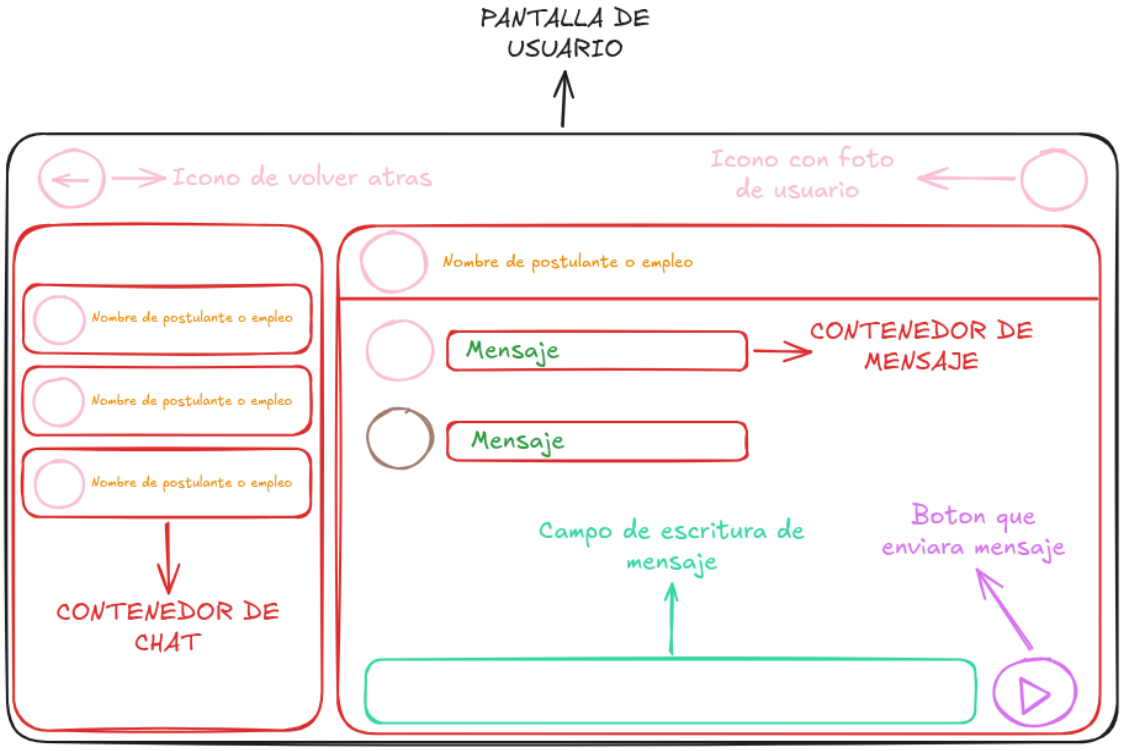


5.7 Funcionalidades Transversales

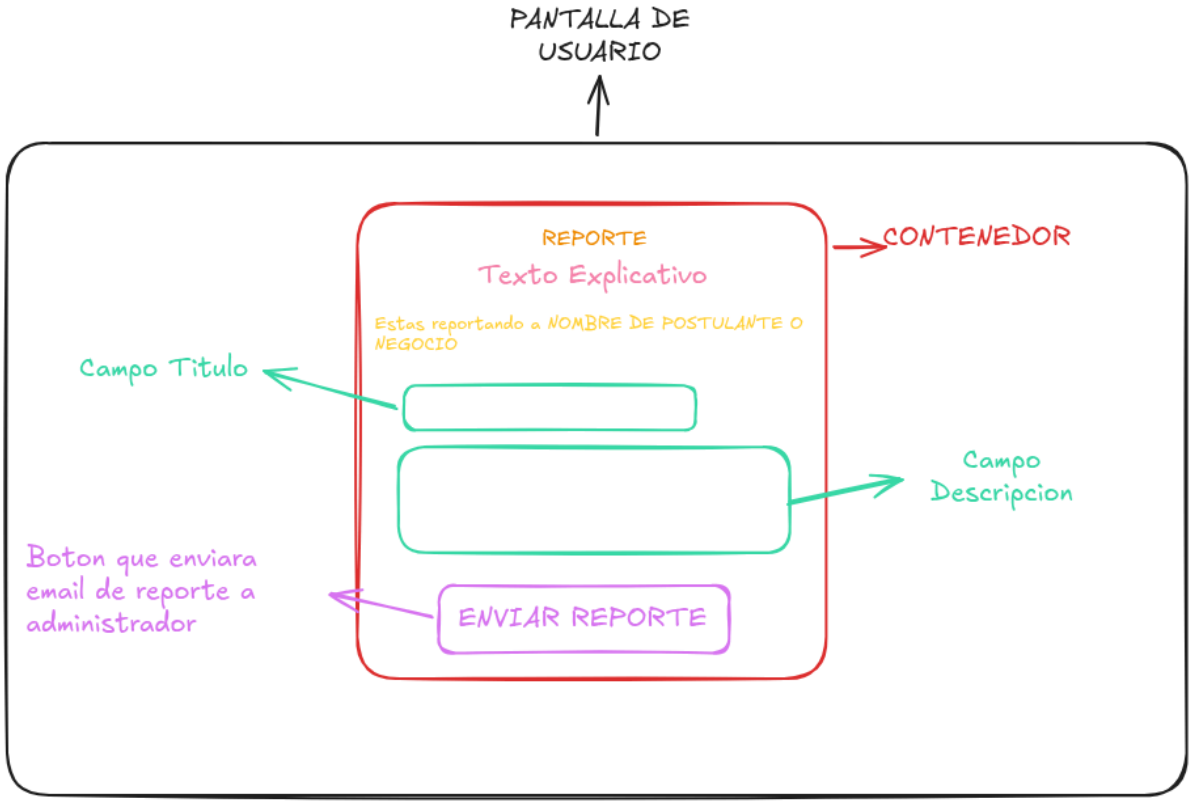
5.7.1 Interfaz de búsqueda



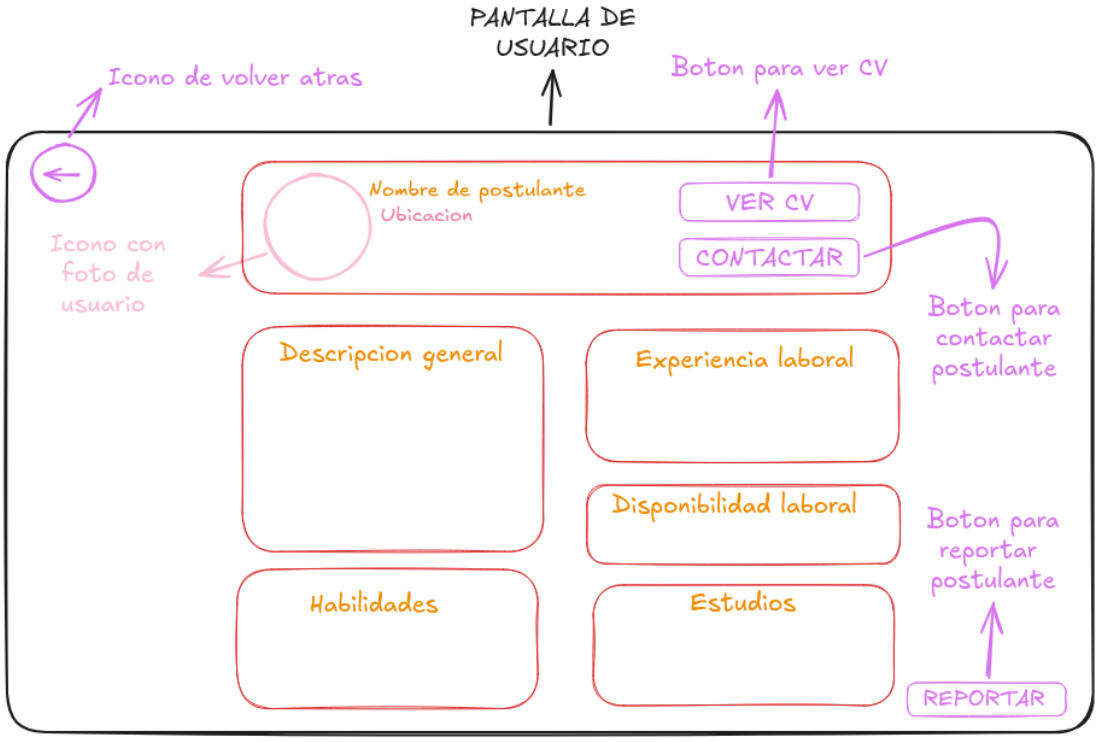
5.7.2 Interfaz de mensajería

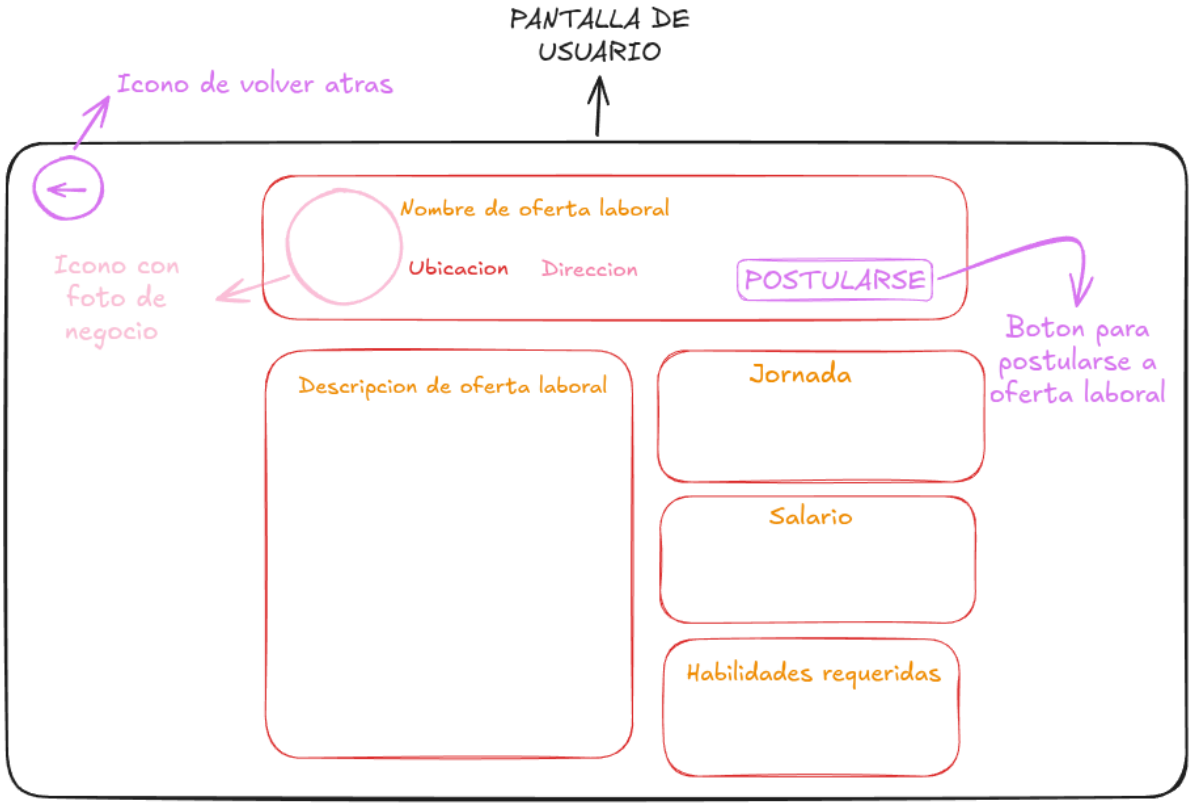


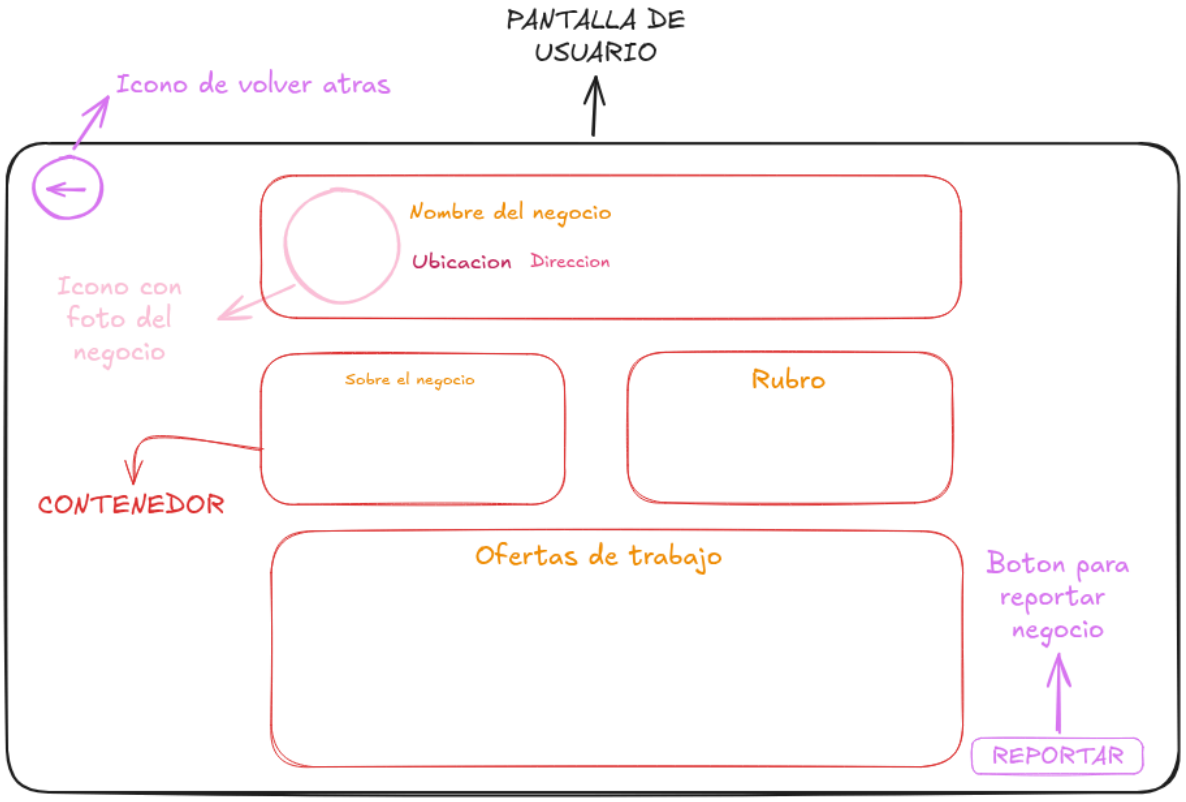
5.7.3 Interfaz de reportes



5.7.4 Visualización de perfiles




PANTALLA DE USUARIO
Icono de volver atras
Nombre de oferta laboral
Ubicacion
Direccion
POSTULARSE
Icono con foto de negocio
Boton para postularse a oferta laboral
Descripcion de oferta laboral
Jornada
Salario
Habilidades requeridas


PANTALLA DE USUARIO
Icono de volver atras
Nombre del negocio
Ubicacion
Direccion
Icono con foto del negocio
Sobre el negocio
Rubro
CONTENEDOR
Ofertas de trabajo
Boton para reportar negocio
REPORTAR

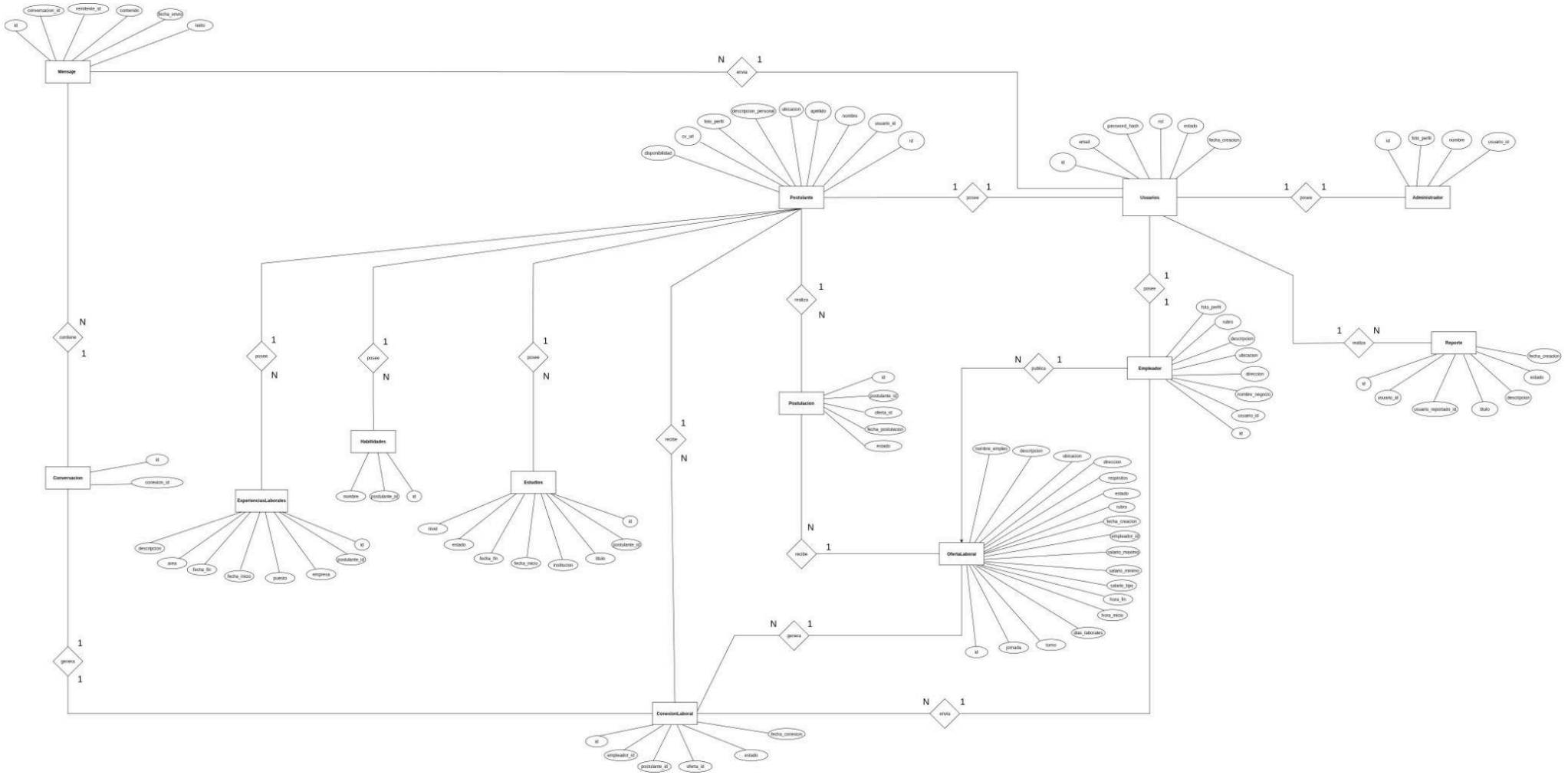
6. BASE DE DATOS

La base de datos fue diseñada con el objetivo de organizar y administrar la información correspondiente a usuarios, postulaciones laborales, ofertas de empleo, conexiones entre usuarios y mensajería interna dentro de la plataforma.

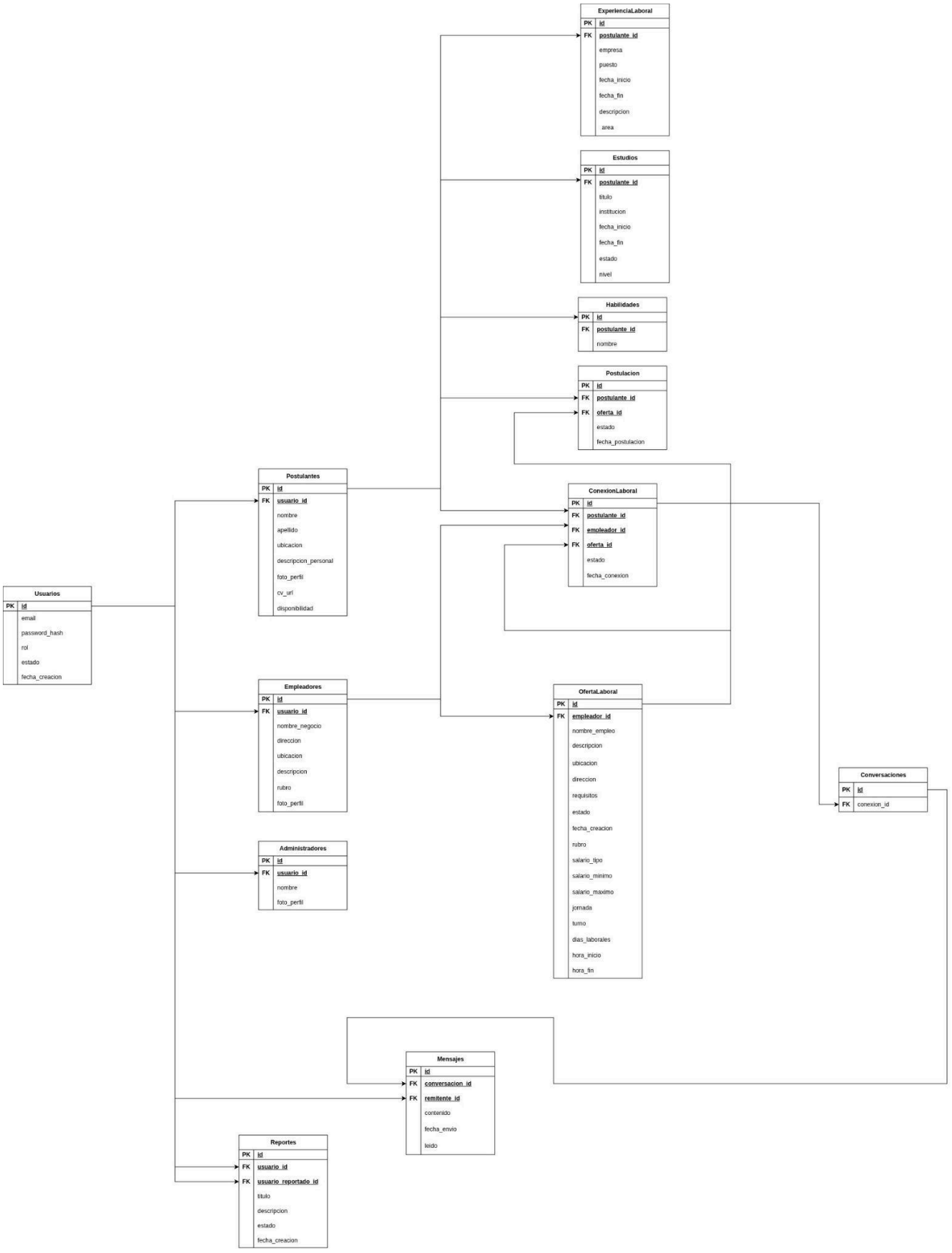
Además, el modelo implementado busca garantizar una estructura clara y escalable, permitiendo mantener la integridad de los datos y facilitar futuras ampliaciones del sistema. Para ello, se definieron relaciones entre las distintas entidades según las funcionalidades desarrolladas en la aplicación, tales como el registro de usuarios, gestión de ofertas laborales, postulaciones, conexiones laborales y conversaciones en tiempo real.

A continuación, se presenta el diagrama entidad-relación general del sistema junto con la descripción de las entidades y relaciones implementadas.

6.1 Diagrama entidad-relación (Conceptual)



6.2 Esquema de datos relacional (Lógico)



6.3 Entidades y tablas

En esta sección se describen las entidades que conforman la base de datos del sistema junto con sus respectivas tablas.

6.3.1 Usuarios

Campo	Tipo de dato	Valores
id	INTEGER	
email	VARCHAR	
password_hash	VARCHAR	
rol	VARCHAR	• POSTULANTE • EMPLEADOR • ADMINISTRADOR
estado	VARCHAR	• ACTIVO • DESACTIVADO • BANEADO
fecha_creacion	TIMESTAMP	

6.3.2 Postulantes

Campo	Tipo de dato	Valores
id	INTEGER	
usuario_id	INTEGER	
nombre	VARCHAR	
apellido	VARCHAR	
ubicacion	VARCHAR	
descripcion_personal	TEXT	
foto_perfil	VARCHAR	
cv_url	VARCHAR	
disponibilidad	VARCHAR	• JORNADA COMPLETA • MEDIA_JORNADA • FINES SEMANA • A CONVENIR

6.3.3 Empleadores

Campo	Tipo de dato	Valores
id	INTEGER	
usuario_id	INTEGER	
nombre_negocio	VARCHAR	
direccion	VARCHAR	
ubicacion	VARCHAR	
descripcion	TEXT	
rubro	VARCHAR	EJ: ● Tecnología ● Gastronomía
foto_perfil	VARCHAR	

6.3.4 Administradores

Campo	Tipo de dato	Valores
id	INTEGER	
usuario_id	INTEGER	
nombre	VARCHAR	
foto_perfil	VARCHAR	

6.3.5 Habilidades

Campo	Tipo de dato	Valores
id	INTEGER	
postulante_id	INTEGER	
nombre	VARCHAR	EJ: ● Python ● Atencion al cliente

6.3.6 Estudios

Campo	Tipo de dato	Valores
id	INTEGER	
postulante_id	INTEGER	
titulo	VARCHAR	
institucion	VARCHAR	
fecha_inicio	DATE	
fecha_fin	DATE	
estado	VARCHAR	EN_CURSO FINALIZADO NO_TERMINADO
nivel	VARCHAR	• PRIMARIO • SECUNDARIO • UNIVERSITARIO • TERCIARIO • CURSO

6.3.7 Experiencias Laborales

Campo	Tipo de dato	Valores
id	INTEGER	
postulante_id	INTEGER	
empresa	VARCHAR	
puesto	VARCHAR	
fecha_inicio	DATE	
fecha_fin	DATE	
descripcion	TEXT	
area	VARCHAR	

6.3.8 Ofertas Laborales

Campo	Tipo de dato	Valores
id	INTEGER	
empleador_id	INTEGER	
titulo	VARCHAR	
descripcion	TEXT	
requisitos	ARRAY	
ubicacion	VARCHAR	
direccion	VARCHAR	
rubro	VARCHAR	
estado	VARCHAR	• ACTIVA • PAUSADA • CERRADA
fecha_creacion	TIMESTAMP	
salario_tipo	VARCHAR	• FIJO • RANGO • A CONVENIR
salario_minimo	NUMERIC(10,2)	
salario_maximo	NUMERIC(10,2)	
jornada	VARCHAR	• COMPLETA • MEDIA JORNADA • TEMPORAL • A CONVENIR
turno	VARCHAR	• MAÑANA • TARDE • NOCHE • A CONVENIR
dias_laborales	ARRAY	
hora_inicio	TIME	
hora_fin	TIME	

6.3.9 Postulaciones

Campo	Tipo de dato	Valores
id	INTEGER	
postulante_id	INTEGER	
empleador_id	INTEGER	
oferta_id	INTEGER	
estado	VARCHAR	• PENDIENTE • ACEPTADA • RECHAZADA
fecha_postulacion	TIMESTAMP	

6.3.10 Conexiones Laborales

Campo	Tipo de dato	Valores
id	INTEGER	
postulante_id	INTEGER	
empleador_id	INTEGER	
oferta_id	INTEGER	
estado	VARCHAR	• PENDIENTE • ACEPTADA • RECHAZADA
fecha_conexion	TIMESTAMP	

6.3.11 Conversaciones

Campo	Tipo de dato	Valores
id	INTEGER	
conexion_laboral_id	INTEGER	

6.3.12 Mensajes

Campo	Tipo de dato	Valores
id	INTEGER	
conversacion_id	INTEGER	
remitente_id	INTEGER	
contenido	TEXT	
leido	BOOLEAN	• True • False
fecha_envio	TIMESTAMP	

6.3.13 Reportes

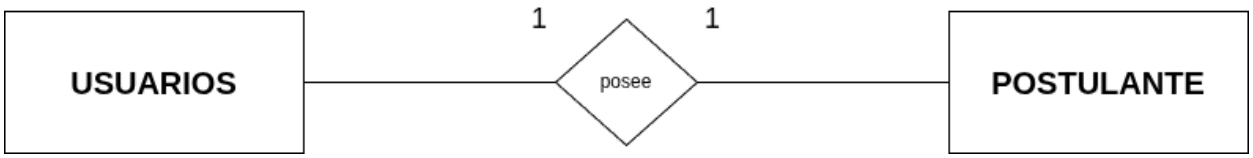
Campo	Tipo de dato	Valores
id	INTEGER	
usuario_id	INTEGER	
usuario_reportado_id	INTEGER	
motivo	VARCHAR	
descripcion	TEXT	
estado	VARCHAR	• PENDIENTE • REVISADO • SANCIONADO
fecha_creacion	TIMESTAMP	

6.4 Relaciones entre tablas

6.4.1 Relaciones de Usuarios

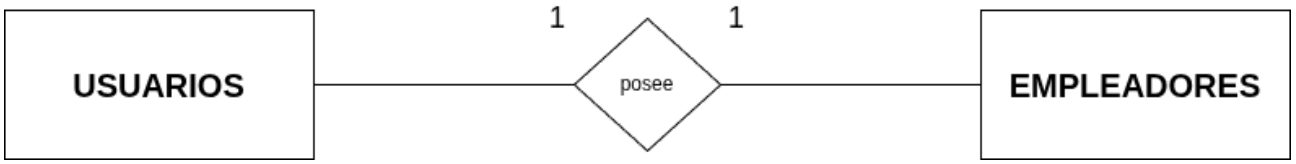
Usuarios y Postulantes (1:1)

La relación entre Usuarios y Postulantes es de uno a uno (1:1). Cada postulante posee una única cuenta de usuario asociada para acceder a la plataforma, mientras que cada usuario con rol de postulante se encuentra vinculado a un único perfil profesional.



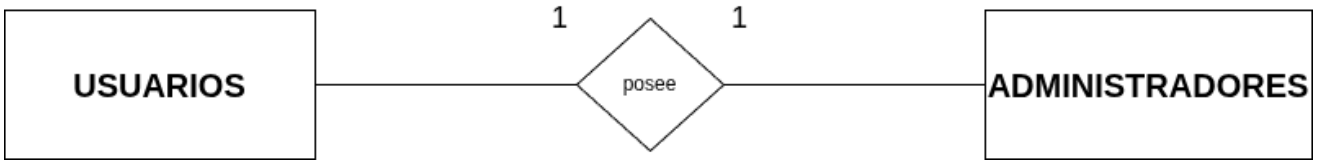
Usuarios y Empleadores (1:1)

La relación entre Usuarios y Empleadores es de uno a uno (1:1). Cada empleador dispone de una única cuenta de usuario asociada, permitiéndole gestionar la información de su organización, publicar ofertas laborales y establecer conexiones con postulantes.



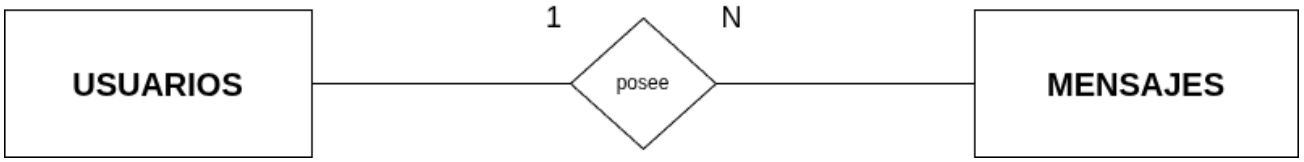
Usuarios y Administradores (1:1)

La relación entre Usuarios y Administradores es de uno a uno (1:1). Cada administrador posee una única cuenta de usuario asociada, permitiendo la gestión y supervisión de las distintas funcionalidades administrativas del sistema.



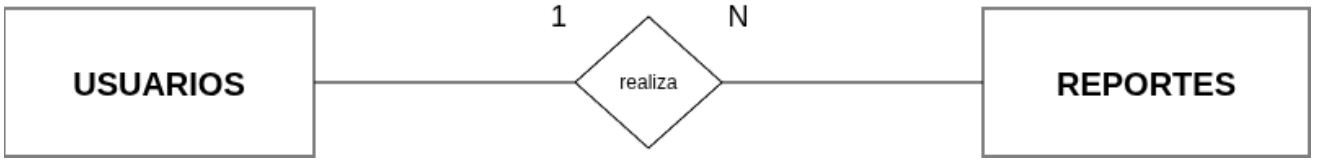
Usuarios y Mensajes (1:N)

La relación entre Usuarios y Mensajes es de uno a muchos (1:N). Un usuario puede enviar múltiples mensajes a través de las conversaciones disponibles dentro de la plataforma, mientras que cada mensaje posee un único remitente.



Usuarios y Reportes (1:N)

La relación entre Usuarios y Reportes es de uno a muchos (1:N). Un usuario puede generar múltiples reportes dentro de la plataforma, mientras que cada reporte es realizado por un único usuario.



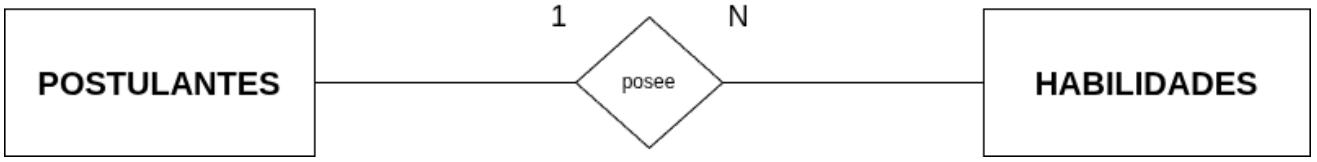
Usuarios y Reportes (Usuario reportado) (1:N)

La relación entre Usuarios y Reportes también permite identificar al usuario reportado. Un usuario puede recibir múltiples reportes realizados por otros usuarios, mientras que cada reporte se encuentra asociado a un único usuario denunciado.

6.4.2 Relaciones de Postulantes

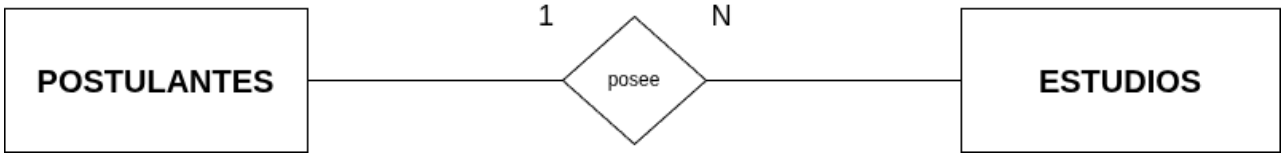
Postulantes y Habilidades (1:N)

Un postulante puede registrar múltiples habilidades dentro de su perfil profesional, mientras que cada habilidad pertenece a un único postulante. Esta relación permite representar las competencias y conocimientos específicos de cada candidato.



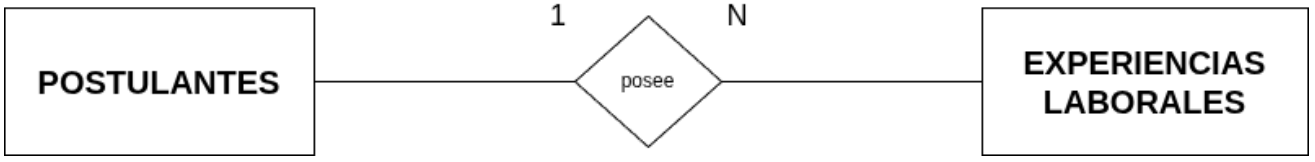
Postulantes y Estudios (1:N)

La relación entre Postulantes y Estudios es de uno a muchos (1:N). Un postulante puede registrar múltiples estudios académicos o formativos, mientras que cada estudio se encuentra asociado a un único postulante.



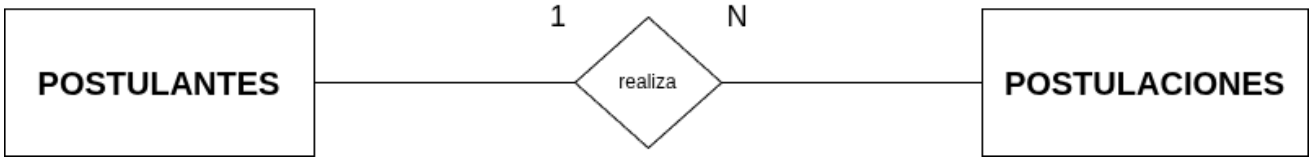
Postulantes y Experiencias Laborales (1:N)

La relación entre Postulantes y ExperienciasLaborales es de uno a muchos (1:N). Un postulante puede registrar múltiples experiencias laborales a lo largo de su trayectoria profesional, mientras que cada experiencia laboral pertenece a un único postulante.



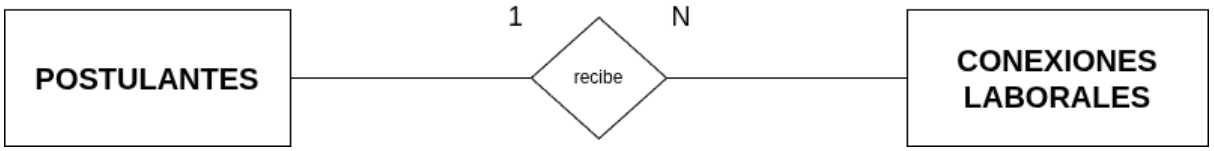
Postulantes y Postulaciones (1:N)

La relación entre Postulantes y Postulaciones es de uno a muchos (1:N). Un postulante puede realizar múltiples postulaciones a distintas ofertas laborales, mientras que cada postulación corresponde a un único postulante.



Postulantes y Conexiones Laborales (1:N)

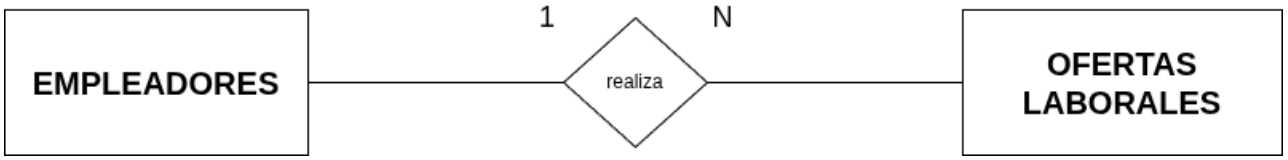
La relación entre Postulantes y ConexionesLaborales es de uno a muchos (1:N). Un postulante puede establecer múltiples conexiones laborales con empleadores, mientras que cada conexión laboral se encuentra asociada a un único postulante.



6.4.3 Relaciones de Empleadores

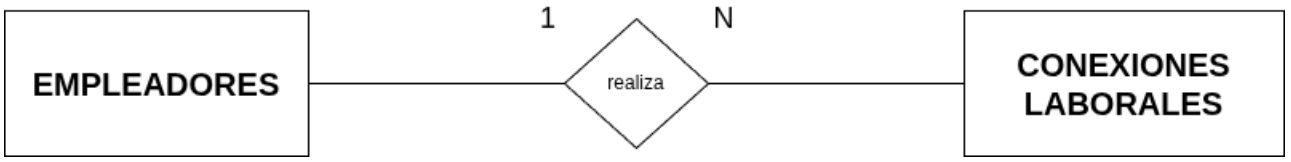
Empleadores y Ofertas Laborales (1:N)

La relación entre Empleadores y OfertasLaborales es de uno a muchos (1:N). Un empleador puede publicar múltiples ofertas laborales dentro de la plataforma, mientras que cada oferta laboral pertenece a un único empleador.



Empleadores y Conexiones Laborales (1:N)

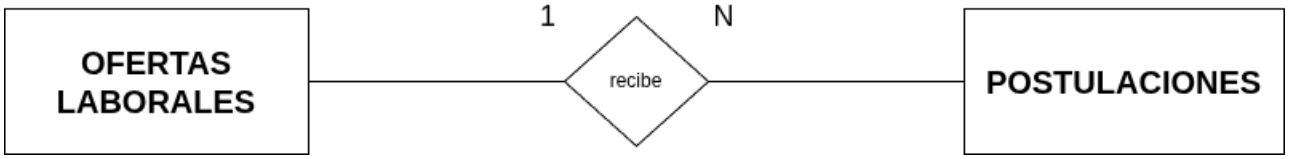
La relación entre Empleadores y ConexionesLaborales es de uno a muchos (1:N). Un empleador puede establecer múltiples conexiones laborales con distintos postulantes, mientras que cada conexión laboral se encuentra asociada a un único empleador.



6.4.4 Relaciones de Ofertas Laborales

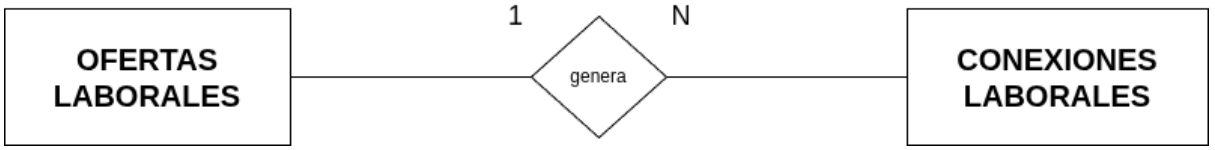
Ofertas Laborales y Postulaciones (1:N)

La relación entre OfertasLaborales y Postulaciones es de uno a muchos (1:N). Una oferta laboral puede recibir múltiples postulaciones por parte de distintos candidatos, mientras que cada postulación se encuentra asociada a una única oferta laboral.



Ofertas Laborales y Conexiones Laborales (1:N)

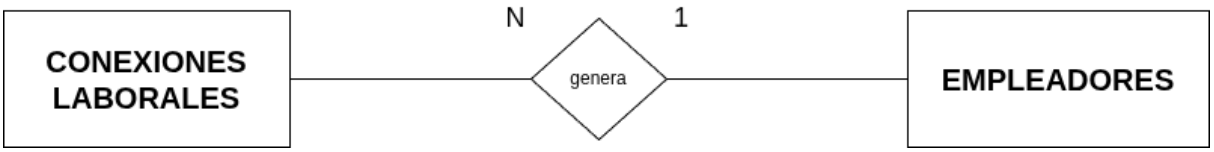
La relación entre OfertasLaborales y ConexionesLaborales es de uno a muchos (1:N). Una oferta laboral puede generar múltiples conexiones laborales con postulantes, mientras que cada conexión laboral se encuentra vinculada a una única oferta laboral.



6.4.5 Relaciones de Conexiones Laborales

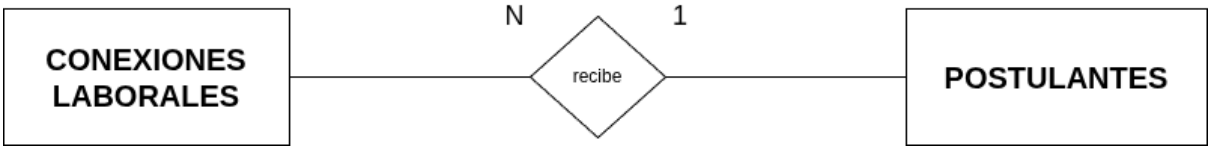
Conexiones Laborales y Empleadores (N:1)

La relación entre ConexionesLaborales y Empleadores es de muchos a uno (N:1). Un empleador puede participar en múltiples conexiones laborales con distintos postulantes, mientras que cada conexión laboral se encuentra asociada a un único empleador.



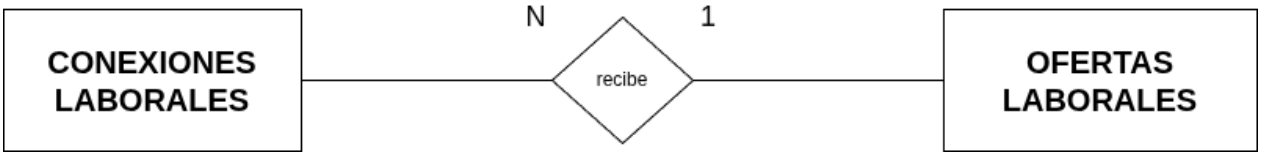
Conexiones Laborales y Postulantes (N:1)

La relación entre ConexionesLaborales y Postulantes es de muchos a uno (N:1). Un postulante puede participar en múltiples conexiones laborales a lo largo de su actividad dentro de la plataforma, mientras que cada conexión laboral pertenece a un único postulante.



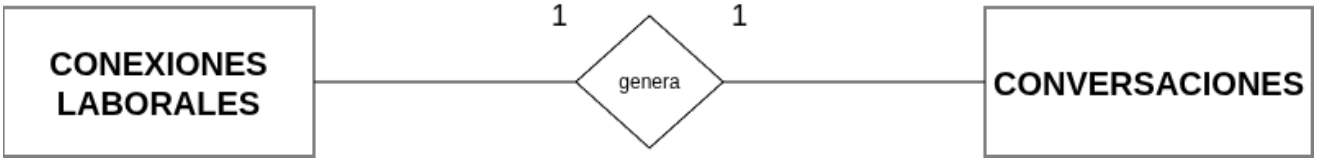
Conexiones Laborales y OfertasLaborales (N:1)

La relación entre ConexionesLaborales y OfertasLaborales es de muchos a uno (N:1). Una conexión laboral puede originarse a partir de una oferta laboral específica, mientras que una oferta laboral puede generar múltiples conexiones laborales con diferentes postulantes.



Conexiones Laborales y Conversaciones (1:1)

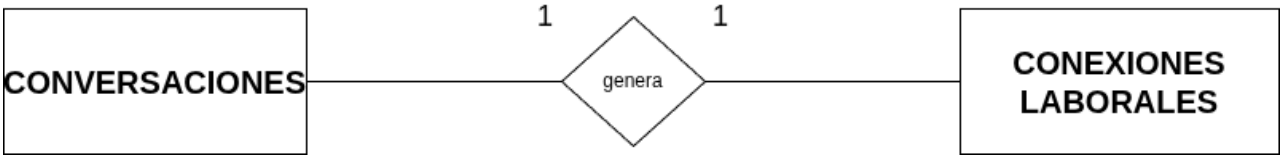
La relación entre ConexionesLaborales y Conversaciones es de uno a uno (1:1). Cada conexión laboral genera una única conversación entre el empleador y el postulante, mientras que cada conversación pertenece exclusivamente a una conexión laboral determinada.



6.4.6 Relaciones de Conversaciones

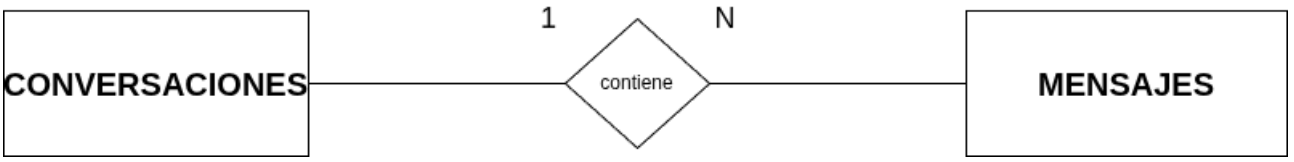
Conversaciones y Conexiones Laborales (1:1)

La relación entre Conversaciones y ConexionesLaborales es de uno a uno (1:1). Cada conversación se encuentra asociada a una única conexión laboral, mientras que cada conexión laboral genera una única conversación entre las partes involucradas.



Conversaciones y Mensajes (1:N)

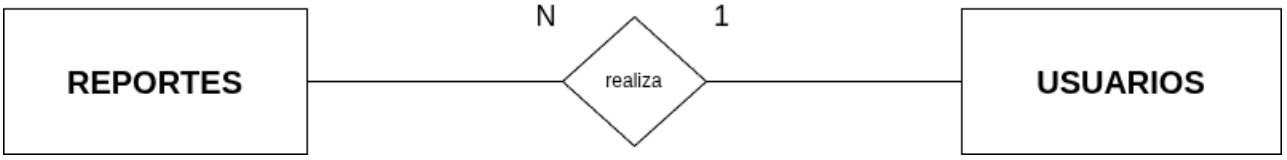
La relación entre Conversaciones y Mensajes es de uno a muchos (1:N). Una conversación puede contener múltiples mensajes intercambiados entre los usuarios, mientras que cada mensaje pertenece a una única conversación.



6.4.7 Relaciones de Reportes

Reportes y Usuarios (Usuario que reporta) (N:1)

La relación entre Reportes y Usuarios es de muchos a uno (N:1). Un usuario puede generar múltiples reportes dentro de la plataforma, mientras que cada reporte es realizado por un único usuario.



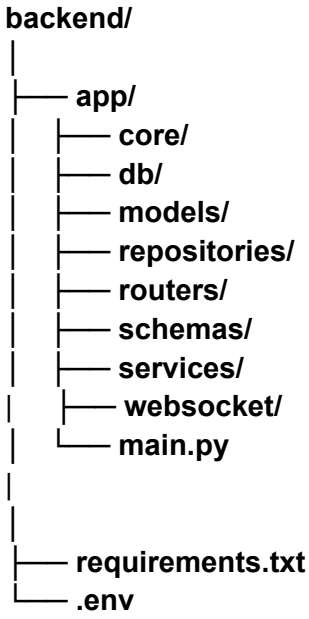
Reportes y Usuarios (Usuario que reporta) (N:1)

La relación entre Reportes y Usuarios también permite identificar al usuario denunciado. Un usuario puede recibir múltiples reportes por parte de otros usuarios, mientras que cada reporte se encuentra asociado a un único usuario reportado.

7. BACKEND

7.1 Estructura del Proyecto

7.1.1 Estructura de directorios



7.1.2 Descripción de directorios

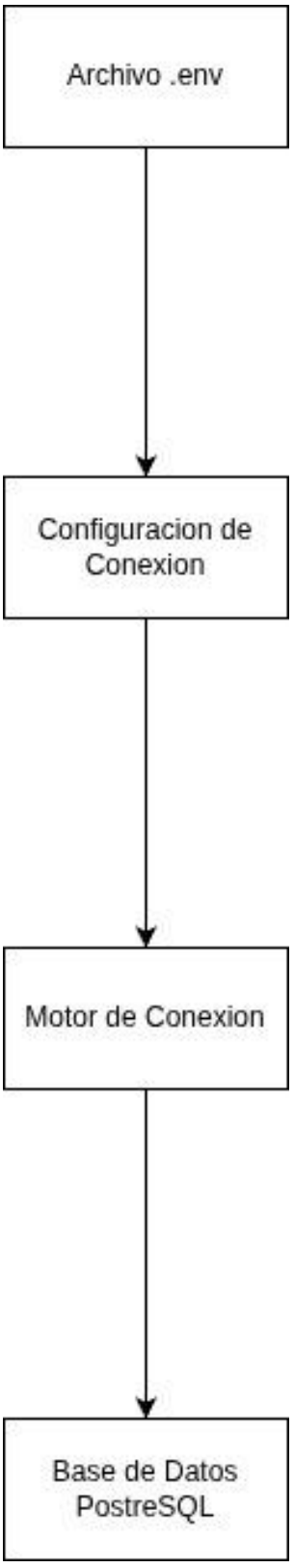
Directorio	Descripción
core/	Contiene componentes compartidos y configuraciones generales del backend.
db/	Administra la conexión y las sesiones con la base de datos.
models/	Define las entidades ORM del sistema.
repositories/	Implementa las operaciones de acceso a datos.
routers/	Define los endpoints de la API REST.
schemas/	Contiene los modelos de validación y serialización de datos.
services/	Implementa la lógica de negocio de la aplicación.
websocket/	Implementa las conexiones WebSocket para la mensajería.
main.py	Inicializa y configura la aplicación backend.

7.2 Persistencia de datos

7.2.1 Conexión con la base de datos

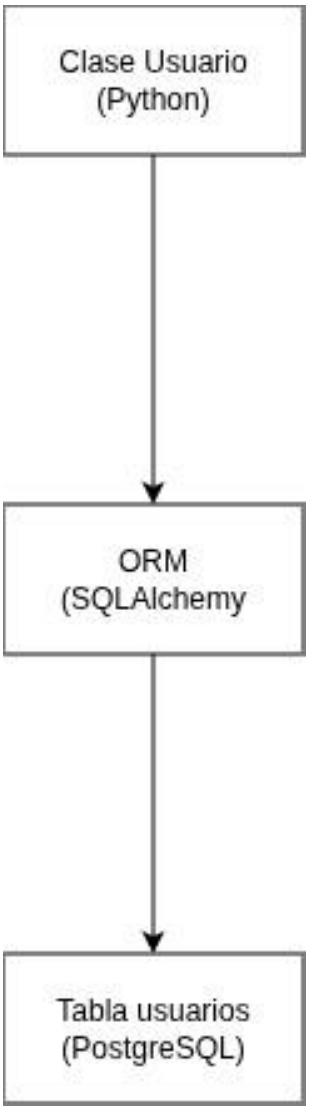
El backend establece la conexión con la base de datos mediante una configuración centralizada que permite acceder a la información necesaria para iniciar la comunicación con el servidor de base de datos. Los parámetros de conexión, como el host, puerto, nombre de la base de datos, usuario y contraseña, se obtienen a partir de variables de entorno, evitando almacenar información sensible directamente en el código fuente.

Una vez obtenida la configuración, se crea una conexión reutilizable que permite a la aplicación interactuar con la base de datos durante su ejecución. Esta conexión constituye el punto de acceso utilizado por los distintos componentes del sistema para realizar operaciones de almacenamiento y consulta de información.



7.2.2 Mapeo objeto-relacional (ORM)

El backend implementa un mapeo objeto-relacional (ORM) para representar las entidades del sistema mediante clases de Python. Esta estrategia permite establecer una correspondencia entre los objetos de la aplicación y las tablas de la base de datos, simplificando la manipulación de la información y reduciendo la necesidad de escribir consultas SQL de forma manual.

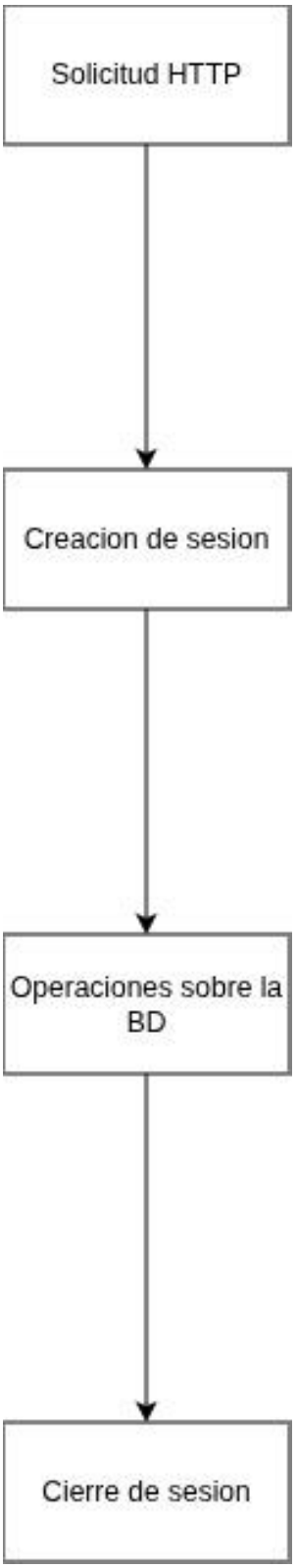


Para llevar a cabo este proceso se utiliza SQLAlchemy, el cual permite definir cada entidad como una clase que representa una tabla de la base de datos, mientras que sus atributos corresponden a las columnas de dicha tabla. De esta manera, las operaciones de creación, consulta, actualización y eliminación de registros se realizan mediante objetos, favoreciendo una implementación más organizada y mantenible.

7.2.3 Gestión de sesiones

La interacción entre el backend y la base de datos se realiza mediante sesiones administradas por SQLAlchemy. Cada sesión representa un contexto de trabajo que permite ejecutar operaciones de consulta y modificación sobre los datos, manteniendo el control de las transacciones realizadas durante el procesamiento de una solicitud.

Las sesiones son creadas cuando la aplicación requiere acceder a la base de datos y permanecen activas únicamente durante el tiempo necesario para completar la operación correspondiente. Una vez finalizada la solicitud, la sesión es cerrada automáticamente, liberando los recursos utilizados y garantizando un uso eficiente de las conexiones disponibles.



7.2.4 Operaciones de persistencia (DAO)

Las operaciones de persistencia son implementadas mediante una capa de repositorios, cuya responsabilidad consiste en gestionar el acceso a la base de datos de forma independiente de la lógica de negocio. Esta separación permite desacoplar las operaciones de almacenamiento y consulta del resto de la aplicación, favoreciendo una arquitectura más organizada y mantenible.

A través de los repositorios se realizan las operaciones fundamentales sobre las entidades del sistema, incluyendo la creación, consulta, actualización y eliminación de registros. De esta manera, los servicios del backend interactúan con la información utilizando una interfaz centralizada, evitando el acceso directo a la base de datos y promoviendo la reutilización del código.

7.3 API REST

7.3.1 FastAPI

La API REST fue implementada mediante FastAPI y constituye el mecanismo de comunicación entre el frontend y el backend de la plataforma. Su función es recibir las solicitudes realizadas por los usuarios, procesarlas mediante la lógica de negocio correspondiente y devolver una respuesta con la información solicitada o el resultado de la operación ejecutada.

La implementación de la API permite centralizar el acceso a las funcionalidades del sistema mediante un conjunto de endpoints organizados por módulos, facilitando la interacción con los distintos recursos de la aplicación de forma estructurada y manteniendo una separación clara entre la interfaz de usuario y la lógica del servidor.

7.3.2 Organización de los Endpoints

Los endpoints de la API fueron organizados de manera modular, agrupando las distintas rutas según la funcionalidad que representan dentro de la plataforma. Esta organización permite mantener una estructura clara y facilita el mantenimiento, la incorporación de nuevas funcionalidades y la reutilización del código.

Cada conjunto de endpoints se encuentra definido en un router independiente, encargado de gestionar las solicitudes relacionadas con un recurso específico del sistema. De esta manera, las operaciones correspondientes a autenticación, gestión de usuarios, ofertas laborales, postulaciones, mensajería y demás funcionalidades permanecen desacopladas entre sí, favoreciendo una arquitectura más ordenada y escalable.

routers/
├── administrador_router.py
├── autenticacion_router.py
├── conversacion_router.py
├── empleador_router.py
├── mensaje_router.py
├── oferta_laboral_router.py
├── postulante_router.py
└── reporte_router.py

7.3.3 Endpoints Principales

Administrador

Método	Endpoint	Descripción
PUT	/administrador/actualizar-reporte	Actualizar estado de reporte
GET	/administrador/buscar-reportes	Buscar reportes a usuarios
GET	/administrador/buscar-usuarios	Buscar usuarios
PUT	/administrador/desactivar-usuario	Desactivar usuario
PUT	/administrador/banear-usuario	Banear usuario

PUT	/administrador/activar-usuario	Activa usuario
GET	/administrador/obtener-postulante-usuario-id	Obtiene al postulante mediante su usuario id
GET	/administrador/obtener-empleador-usuario-id	Obtiene al empleador mediante su usuario id

Autenticación

Método	Endpoint	Descripción
POST	/autenticacion/registrar-postulante	Registrar postulante
POST	/autenticacion/registrar-empleador	Registrar empleador
POST	/autenticacion/login	Iniciar sesión en la app
POST	/autenticacion/resetear-contraseña	Recuperar contraseña de usuario
PUT	/autenticacion//restablecer-contraseña	Restablece la contraseña del usuario

Conversación

Método	Endpoint	Descripción
GET	/conversacion/traer-conversaciones-postulante	Obtener conversaciones de postulante
GET	/conversacion/traer-conversaciones-empleador	Obtener conversaciones de empleador

Empleador

Método	Endpoint	Descripción
GET	/empleador/perfil-empleador	Obtener perfil de empleador
GET	/empleador/buscar-empleadores	Buscar empleadores
PUT	/empleador/actualizar-empleador	Actualizar datos de empleador
POST	/empleador/crear-oferta-laboral	Crear oferta laboral

GET	/empleador/filtrar-postulaciones-oferta-laboral	Filtrar las postulaciones de una oferta laboral específica de empleador
GET	/empleador//verificar-conexion-oferta-laboral	Verifica si un postulante posee una conexión labora conl oferta laboral de empleador
GET	/empleador/filtrar-ofertas-laborales	Filtrar ofertas laborales de empleador
PUT	/empleador/actualizar-oferta-laboral	Actualizar datos de oferta laboral
POST	/empleador/crear-conexion-laboral	Crear una conexión laboral mediante solicitud de contacto a postulante
POST	/empleador/crear-conexion-laboral-postulacion	Crear conexion laboral mediante la aceptación de una postulación de postulante
GET	/empleador/buscar-conexiones-laborales	Buscar conexiones laborales de empleadores
GET	/empleador/verificar-conexion-laboral	Verificar si existe una conexión laboral de empleador
PUT	/empleador/actualizar-estado-postulacion	Actualiza el estado de una postulación de oferta laboral
GET	/empleador/{empleador_id}	Obtiene el perfil público de un empleador por su id

Mensaje

Método	Endpoint	Descripción
WEBSOCKET	/mensajes/{conversacion_id}	Conectar chat entre postulante y empleador
GET	/mensajes/listar-mensajes/{conversacion_id}	Listar mensajes de conversación

PUT	/mensajes/actualizar-estado-mensajes	Actualizar estado de mensajes
GET	/mensajes/mensajes-no-leidos	Verificar si el usuario tiene mensajes no leídos

Oferta Laboral

Método	Endpoint	Descripción
GET	/oferta-laboral/buscar-ofertas-laborales	buscar ofertas laborales
GET	/oferta-laboral/{oferta_laboral_id}	Obtiene el detalle de una oferta laboral por su id

Postulante

GET	/postulante/perfil	Obtener perfil de postulante
GET	/postulante/buscar-postulantes	Buscar postulantes
PUT	/postulante/actualizar-postulante	Actualizar los datos de postulante
POST	/postulante/crear-estudio	Crear estudio de postulante
PUT	/postulante/actualizar-estudio	Actualizar datos del estudio de postulante
DELETE	/postulante/eliminar-estudio	Eliminar estudio de postulante
POST	/postulante/crear-experiencia-laboral	Crear experiencia laboral de postulante
PUT	/postulante/actualizar-experiencia-laboral	Actualizar experiencia laboral de postulante
DELETE	/postulante/eliminar-experiencia-laboral	Eliminar experiencia laboral de postulante
POST	/postulante/crear-habilidad	Crear habilidad de postulante
DELETE	/postulante/eliminar-habilidad	Eliminar habilidad de postulante
GET	/postulante/mis-postulaciones	Obtener las postulaciones del postulante

POST	/postulante/crear-postulacion	Crear postulación a oferta laboral
GET	/postulante/verificar-postulacion	Verificar si un postulante está postulado a una oferta laboral
PUT	/postulante/actualizar-solicitud-laboral	Actualizar estado de solicitud laboral con empleador
GET	/postulante/verificar-conexion-laboral	Verificar si existe una conexión laboral
GET	/postulante/buscar-solicitudes-laborales	Buscar conexiones laborales
GET	/postulante/{postulante_id}	Obtiene el perfil público de un postulante por su id

Reportes

Método	Endpoint	Descripción
POST	/reporte/crear-reporte	Crear un reporte

7.3.4 Formato de intercambio de datos

La comunicación entre el frontend y el backend se realiza mediante el formato JSON, el cual permite representar la información de manera estructurada y facilitar el intercambio de datos entre ambos componentes del sistema.

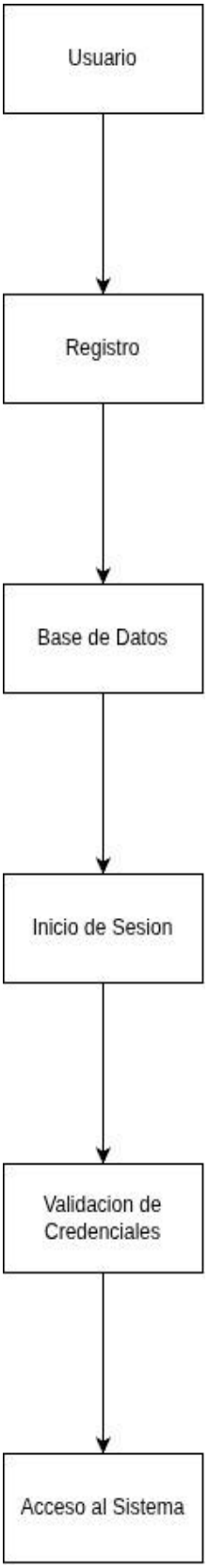
Las solicitudes enviadas desde el frontend utilizan este formato para transmitir la información necesaria hacia el backend, mientras que las respuestas generadas por la API también son devueltas en formato JSON. Durante el procesamiento de cada solicitud, FastAPI utiliza modelos definidos con Pydantic para validar automáticamente los datos recibidos, garantizando que la información cumpla con la estructura y los tipos de datos esperados por la aplicación antes de ser procesada por la lógica de negocio.

7.4 Autenticación y Autorización

7.4.1 Registro e inicio de sesión

El sistema permite que los usuarios creen una cuenta mediante un proceso de registro en el que se almacenan los datos necesarios para su identificación dentro de la plataforma. Durante este proceso, cada usuario recibe un rol que determina las funcionalidades a las que podrá acceder posteriormente.

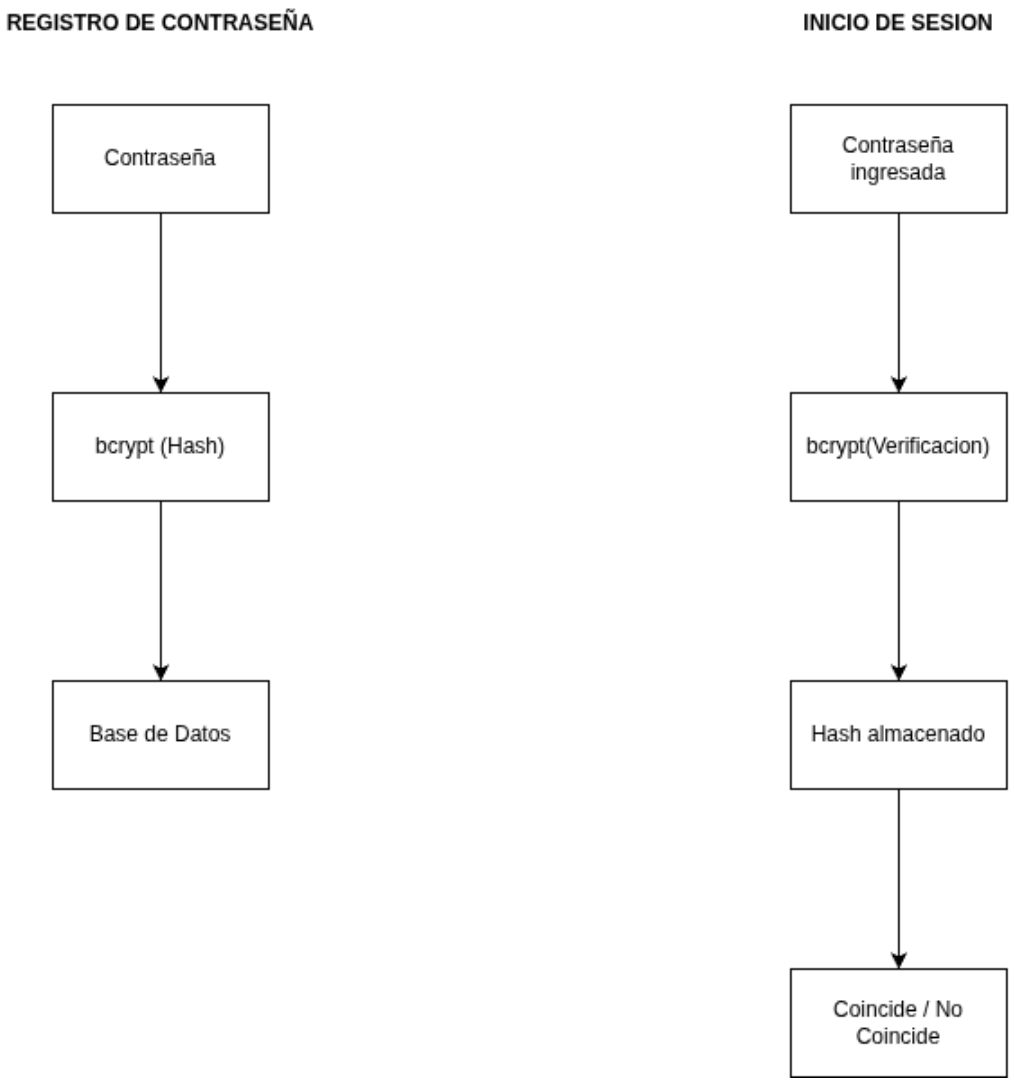
Una vez registrado, el usuario puede iniciar sesión utilizando sus credenciales de acceso. El backend verifica la información proporcionada y, en caso de ser válida, autoriza el acceso a la plataforma y habilita la utilización de los servicios correspondientes a su perfil.



7.4.2 Gestión de contraseñas

Con el objetivo de proteger la información de los usuarios, las contraseñas no son almacenadas en texto plano dentro de la base de datos. Antes de ser persistidas, son sometidas a un proceso de hash mediante la librería bcrypt, generando una representación cifrada que impide recuperar la contraseña original.

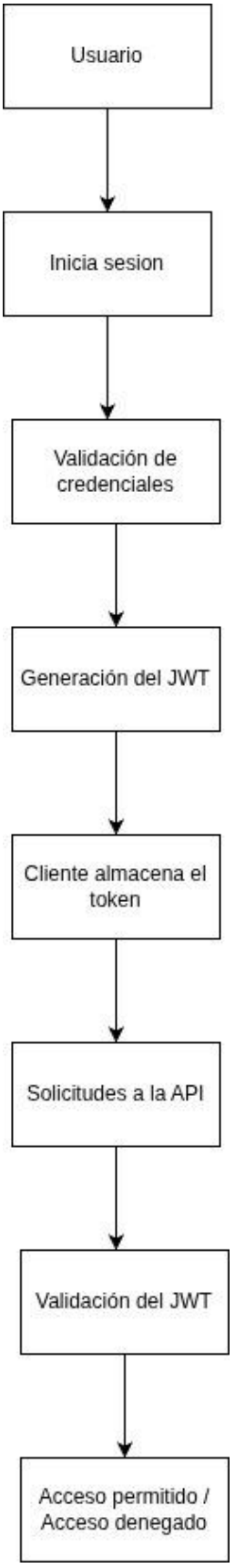
Durante el proceso de autenticación, la contraseña ingresada por el usuario es comparada con el hash almacenado utilizando el mismo algoritmo de verificación. De esta manera, el sistema puede validar la identidad del usuario sin necesidad de conocer ni almacenar su contraseña en formato legible, incrementando la seguridad de la plataforma frente a posibles accesos no autorizados.



7.4.3 Autenticación mediante JWT

La autenticación de los usuarios fue implementada mediante FastAPI utilizando JSON Web Tokens (JWT) como mecanismo de verificación de identidad. Una vez que el usuario ingresa correctamente sus credenciales, el backend genera un token de autenticación, el cual es utilizado durante el resto de la sesión para acceder a los recursos protegidos de la plataforma.

Este token contiene la información necesaria para identificar al usuario y es enviado por el cliente en cada solicitud hacia los endpoints que requieren autenticación.



7.4.4 Control de acceso por roles

El sistema implementa un mecanismo de autorización basado en roles, permitiendo restringir el acceso a las distintas funcionalidades de acuerdo con el tipo de usuario autenticado. Cada cuenta registrada posee un rol específico que determina las operaciones que puede realizar dentro de la plataforma.

Antes de ejecutar una operación protegida, el backend verifica el rol asociado al usuario autenticado y determina si posee los permisos necesarios para acceder al recurso solicitado.

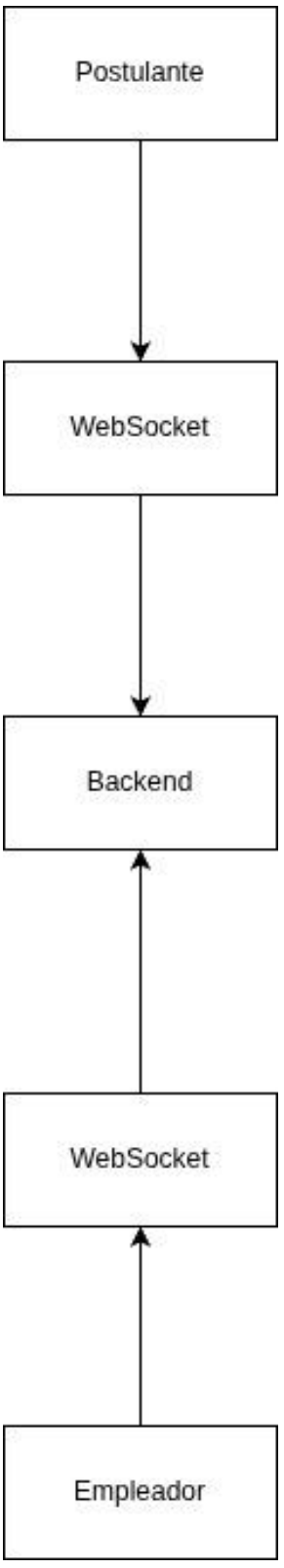
ROL	**FUNCIONALIDADES PRINCIPALES**
Postulante	Gestionar su perfil, postularse a ofertas laborales, administrar conexiones y conversaciones.
Empleador	Publicar ofertas, gestionar postulaciones, establecer conexiones laborales y comunicarse con postulantes.
Administrador	Administrar usuarios, gestionar reportes y supervisar el funcionamiento general de la plataforma.

7.5 Comunicación en tiempo real

7.5.1 Implementación mediante WebSocket

La funcionalidad de mensajería en tiempo real fue implementada utilizando el protocolo WebSocket, el cual permite mantener una conexión persistente entre el cliente y el servidor durante toda la comunicación. A diferencia de las solicitudes HTTP tradicionales, este mecanismo posibilita el intercambio bidireccional de información sin necesidad de establecer una nueva conexión para cada mensaje enviado.

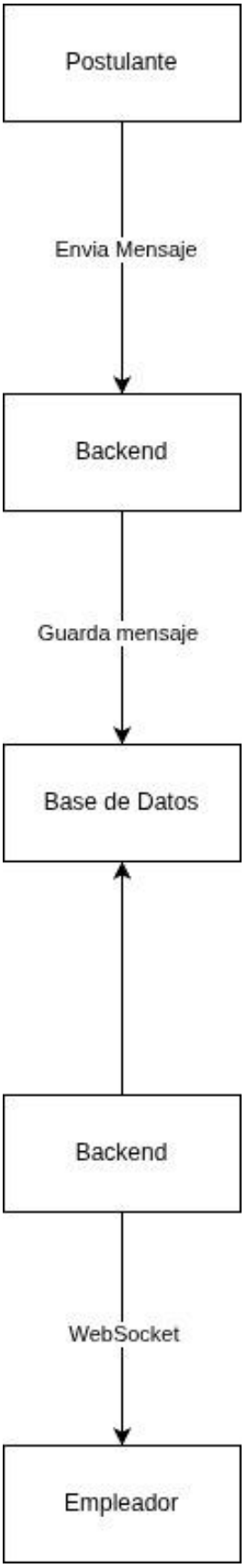
Cuando dos usuarios mantienen una conversación activa, el backend administra la conexión WebSocket correspondiente y distribuye los mensajes hacia el destinatario de manera inmediata. Esta implementación reduce la latencia en la comunicación y proporciona una experiencia de usuario más fluida, permitiendo que los mensajes sean visualizados en tiempo real dentro de la plataforma.



7.5.2 Flujo de comunicación

El proceso de comunicación comienza cuando un usuario envía un mensaje desde la interfaz de la plataforma. El backend recibe la solicitud, valida la información y almacena el mensaje en la base de datos para conservar el historial de la conversación.

Una vez persistido el mensaje, el servidor lo transmite mediante la conexión WebSocket al usuario destinatario, quien lo recibe y visualiza de forma inmediata dentro de la conversación correspondiente. Este flujo garantiza la sincronización de la información entre ambos participantes y permite mantener una comunicación continua sin necesidad de realizar solicitudes repetitivas al servidor.



7.6 Servicios Externos

7.6.1 Cloudinary

Cloudinary es utilizado como servicio de almacenamiento para los archivos multimedia de la plataforma. A través de esta integración se gestionan las imágenes de perfil de los usuarios y los documentos PDF correspondientes a los currículums, evitando almacenar estos archivos directamente en el servidor del backend.

Cada archivo cargado por el usuario es enviado a Cloudinary, el cual genera una dirección única que posteriormente es almacenada en la base de datos. De esta manera, el sistema únicamente administra las referencias a los archivos, mientras que el almacenamiento y la distribución de los mismos son gestionados por el servicio externo.

7.6.2 Brevo

Brevo es el servicio utilizado para el envío de correos electrónicos desde la plataforma. Su principal función es gestionar el proceso de recuperación de contraseña, permitiendo que los usuarios reciban instrucciones para restablecer el acceso a sus cuentas de forma segura.

8. FRONTEND

8.1 Arquitectura del frontend

8.1.1 Estructura del Proyecto

```
frontend/
├── src/
│   ├── components/
│   ├── assets/
|   ├── core/
│   ├── services/
│   ├── views/
│   └── main.py
│
├── requirements.txt
├── .env
```

8.1.2 Descripción de directorios

Directorio	Descripción
assets/	Contiene los recursos estáticos utilizados por la interfaz, como imágenes, íconos y demás archivos gráficos.
components/	Agrupa los componentes reutilizables que conforman la interfaz de usuario, permitiendo mantener un diseño consistente en las distintas vistas de la aplicación.
core/	Contiene el archivo de configuración del frontend, encargado de centralizar la carga y administración de las variables de entorno y demás parámetros generales de la aplicación.
services/	Implementa la comunicación con el backend mediante el consumo de la API REST y otros servicios requeridos por la aplicación.

views/	Define las diferentes pantallas o vistas que componen la interfaz de usuario de la plataforma.
.env	Almacena las variables de entorno utilizadas por la aplicación, evitando incluir información sensible dentro del código fuente.
main.py	Constituye el punto de entrada del frontend, donde se inicializa la aplicación y se configura su funcionamiento general.
requirements.txt	Especifica las dependencias necesarias para la ejecución del proyecto.

8.2 Comunicación con el Backend

La aplicación cliente, desarrollada con Flet (Python), se comunica con el backend mediante una API REST y, para el chat, mediante WebSocket. Este apartado describe el lado del cliente: cómo se arman y envían las peticiones, cómo se manejan los errores y cómo se gestiona la sesión. El diseño de la API y la lógica del servidor se documentan en la sección de Backend.

Se emplean dos mecanismos de comunicación:

- **API REST sobre HTTP con JSON**, para consultas y operaciones puntuales: búsquedas, filtros, altas y cambios de estado.
- **WebSocket**, para el chat entre postulantes y empleadores, donde los mensajes deben llegar en tiempo real.

8.2.1 Capas del cliente

Capa	Responsabilidad	Ejemplos
Vistas	Interfaz, validación de formularios y retroalimentación al usuario	• GestionPostulantesView • MensajeriaView
Servicios del cliente	Un método estático asíncrono por operación; traducen una acción del usuario en una llamada a un endpoint	• AutenticacionService • PostulanteService • EmpleadorService • OfertaLaboralService • ConversacionService • MensajeService • ChatService • ReporteService • AdministradorService
ApiService	Acceso HTTP centralizado: arma la URL, adjunta las credenciales, ejecuta la petición y traduce los errores	• ApiService.get • ApiService.post • ApiService.put • ApiService.delete
Sesion	Datos de la sesión del usuario	• Sesion.iniciar • Sesion.token • Sesion.rol • Sesion.obtener_usuario_id
Configuración	Dirección base del backend; también se usa para construir la URL del WebSocket	• API_URL

8.2.2 Consumo de la API REST

8.2.2.1 Patrón de servicios

Cada operación del backend se expone en el cliente como un método estático asíncrono dentro de un servicio del dominio. El método recibe la página de Flet y los parámetros de la operación, y delega en ApiService. Ejemplo:

```
@staticmethod
    async def filtrar_postulaciones(
        page: ft.Page,
        oferta_id: int | None = None,
        busqueda: str | None = None,
        offset: int = 0
    ):
        """Metodo para filtrar postulaciones de oferta laboral de
empleador"""
        params = {
            "oferta_id": oferta_id,
            "busqueda": busqueda,
            "offset": offset
        }
        return await ApiService.get(
            page=page,
            endpoint="/empleador/filtrar-postulaciones-oferta-laboral",
            params=params
        )
```

Los nombres de los parámetros coinciden con los que declara el endpoint. El identificador del usuario no se envía: el backend lo obtiene del token.

8.2.2.1 Endpoints principales

Autenticación | AutenticacionService

MÉTODO	ENDPOINT	DATOS	USO
POST	/autenticacion/registrar-postulante	cuerpo: usuario (email, password) y postulante (nombre, apellido, ubicacion)	Registro de postulante
POST	/autenticacion/registrar-empleador	cuerpo: usuario y empleador (nombre_negocio, ubicacion, direccion)	Registro de empleador
POST	/autenticacion/login/	cuerpo: email, password	Inicio de sesión; devuelve access_token, rol y exp
POST	/autenticacion/resetear-contraseña	cuerpo: email	Solicitud de recuperación de contraseña
PUT	/autenticacion/restablecer-contraseña	query: token · cuerpo: nueva_contrasena	Nueva contraseña

Postulante | PostulanteService

MÉTODO	ENDPOINT	DATOS	USO
GET	/postulante/perfil	–	Perfil del postulante autenticado
GET	/postulante/{postulante_id}	–	Perfil de un postulante
GET	/postulante/buscar-postulantes	busqueda, ubicacion, estudios, experiencias_laborales, disponibilidad, habilidades, offset	Búsqueda de postulantes

PUT	/postulante/actualizar-postulante	cuerpo: nombre, apellido, ubicacion, descripcion_personal, disponibilidad, foto_perfil, cv	Edición del perfil
POST	/postulante/crear-estudio	cuerpo: titulo, institucion, fecha_inicio, fecha_fin, estado, nivel	Alta de estudio
PUT	/postulante/actualizar-estudio	query: estudio_id · cuerpo: los mismos campos	Edición de estudio
DELETE	/postulante/eliminar-estudio	estudio_id	Baja de estudio
POST	/postulante/crear-experiencia-laboral	cuerpo: empresa, puesto, fecha_inicio, fecha_fin, descripcion, area	Alta de experiencia laboral
PUT	/postulante/actualizar-experiencia-laboral	query: experiencia_laboral_id · cuerpo: los mismos campos	Edición de experiencia laboral
DELETE	/postulante/eliminar-experiencia-laboral	experiencia_laboral_id	Baja de experiencia laboral
POST	/postulante/crear-habilidad	cuerpo: nombre	Alta de habilidad
DELETE	/postulante/eliminar-habilidad	habilidad_id	Baja de habilidad
GET	/postulante/mis-postulaciones	busqueda, offset	Postulaciones del postulante
POST	/postulante/crear-postulacion	cuerpo: oferta_id	Postularse a una oferta
GET	/postulante/verificar-postulacion	oferta_id	Consulta si ya está postulado
PUT	/postulante/actualizar-solicitud-laboral	cuerpo: conexion_laboral_id, estado	Aceptar o rechazar una solicitud de contacto

GET	/postulante/verificar-conexion-laboral	oferta_id	Consulta si existe una conexión con la oferta
GET	/postulante/buscar-solicitudes-laborales	busqueda, offset	Solicitudes de contacto recibidas

Empleador | EmpleadorService

MÉTODO	ENDPOINT	DATOS	USO
GET	/empleador/perfil-empleador	–	Perfil del empleador autenticado
GET	/empleador/{empleador_id}	–	Perfil de un empleador
GET	/empleador/buscar-empleadores	busqueda, direccion, ubicacion, rubro, offset	Búsqueda de empleadores
PUT	/empleador/actualizar-empleador	cuerpo: nombre_negocio, direccion, ubicacion, descripcion, rubro, foto_perfil	Edición del perfil
POST	/empleador/crear-oferta-laboral	cuerpo: titulo, descripcion, requisitos, ubicacion, direccion, rubro, estado, salario_tipo, salario_minimo, salario_maximo, jornada, turno, dias_laborales, hora_inicio, hora_fin	Alta de oferta laboral
PUT	/empleador/actualizar-oferta-laboral	query: oferta_laboral_id · cuerpo: los mismos campos	Edición de oferta laboral
GET	/empleador/filtrar-ofertas-laborales	busqueda	Ofertas del empleador

GET	/empleador/filtrar-postulaciones-oferta-laboral	oferta_id, busqueda, offset	Postulaciones a sus ofertas
PUT	/empleador/actualizar-estado-postulacion	query: oferta_id, postulacion_id · cuerpo: estado	Aceptar o rechazar una postulación
GET	/empleador/verificar-conexion-oferta-laboral	postulante_id	Ofertas sin conexión con un postulante
POST	/empleador/crear-conexion-laboral	cuerpo: oferta_laboral_id, postulante_id (y estado al aceptar una postulación)	Solicitud de contacto a un postulante
GET	/empleador/buscar-conexiones-laborales	busqueda, oferta_id, offset	Solicitudes de contacto enviadas
GET	/empleador/verificar-conexion-laboral	postulante_id, oferta_id	Consulta si existe conexión con un postulante

Ofertas laborales | OfertaLaboralService

MÉTODO	ENDPOINT	DATOS	USO
GET	/oferta-laboral/buscar-ofertas-laborales	busqueda, rubro, jornada, dias_laborales, offset	Búsqueda de ofertas
GET	/oferta-laboral/{oferta_laboral_id}	–	Detalle de una oferta

Conversación-mensaje |ConversacionService-MensajeService-ChatService

MÉTODO	ENDPOINT	DATOS	USO
GET	/conversacion/traer-conversaciones-postulante	–	Conversaciones del postulante
GET	/conversacion/traer-conversaciones-empleador	oferta_id	Conversaciones del empleador, filtradas por oferta
GET	/mensajes/listar-mensajes/{conversacion_id}	–	Historial de una conversación
PUT	/mensajes/actualizar-estado-mensajes	cuerpo: conversacion_id	Marcar mensajes como leídos
GET	/mensajes/mensajes-no-leidos	–	Consulta si hay mensajes sin leer
WebSocket	/mensajes/{conversacion_id}?token=token	mensajes JSON con contenido	Chat en tiempo real

Reportes | ReporteService

MÉTODO	ENDPOINT	DATOS	USO
POST	/reporte/crear-reporte	query: usuario_reportado_id · cuerpo: motivo, descripcion	Reportar a un usuario

Administración | AdministradorService

MÉTODO	ENDPOINT	DATOS	USO
GET	/administrador/buscar-usuarios	filtro, busqueda, offset	Búsqueda de usuarios

PUT	/administrador/desactivar-usuario	query: desactivar_usuario_id	Desactivar un usuario
PUT	/administrador/banear-usuario	query: banear_usuario_id	Banear un usuario
PUT	/administrador/activar-usuario	query: activar_usuario_id	Reactivar un usuario
GET	/administrador/buscar-reportes	filtro, busqueda, offset	Búsqueda de reportes
PUT	/administrador/actualizar-reporte	query: reporte_id · cuerpo: estado	Marcar un reporte como revisado o sancionado
GET	/administrador/obtener-postulante-usuario-id	usuario_id	Postulante asociado a un usuario

8.2.2.1 Manejo de errores

ApiService traduce las respuestas de error en excepciones propias definidas en el módulo api_excepciones. Las vistas las capturan con try/except y deciden cómo informar al usuario.

Excepción	Situación	Comportamiento de la interfaz
ErrorAutenticacion	Credenciales incorrectas en el inicio de sesión, o token inválido o vencido	En el login muestra Email o contraseña incorrectos. En el resto de las vistas muestra el diálogo Sesión expirada y, tras 5 segundos, redirige a /login
ErrorValidacion	El backend rechaza los datos enviados	Muestra Los datos ingresados no son válidos
ErrorNoEncontrado	El recurso solicitado no existe	Muestra un mensaje específico, por ejemplo No se encontró el postulante solicitado
ErrorServidor	Falla del servidor o error inesperado	Muestra un mensaje que invita a reintentar

8.2.3 Gestión de autenticación

8.2.3.1 Inicio de sesión

1. El usuario ingresa email y contraseña. La vista válida en el cliente que el email contenga @ y que la contraseña tenga al menos 8 caracteres.
2. AutenticacionService.iniciar_sesion(page, email, contrasena) envía las credenciales con POST /autenticacion/login/.
3. Si son correctas, el backend responde con access_token, rol y exp (la fecha de expiración, en formato ISO 8601).
4. El servicio guarda esos tres datos con Sesion.iniciar(page, token, rol, exp).
5. La vista consulta Sesion.rol(page) y redirige: los postulantes y empleadores van a /busqueda; los administradores, a /perfil-personal-administrador.
6. Si las credenciales son incorrectas se lanza ErrorAutenticacion y se informa al usuario sin revelar cuál de los dos datos falló.

8.2.3.2 Almacenamiento de la sesión

La clase Sesion concentra los datos del usuario autenticado y los expone mediante métodos asociados a la página de Flet:

MÉTODO	FUNCION
Sesion.iniciar(page, token, rol, exp)	Guarda el token, el rol y la fecha de expiración al iniciar sesión
Sesion.token(page)	Devuelve el token de acceso del usuario
Sesion.rol(page)	Devuelve el rol: POSTULANTE, EMPLEADOR o ADMINISTRADOR
Sesion.obtener_usuario_id(page)	Devuelve el identificador del usuario, usado por ejemplo para distinguir los mensajes propios en el chat

8.2.3.3 Envío del token

Cada petición a un endpoint protegido lleva el token de la sesión. A verificar: que ApiService lo adjunte en el encabezado Authorization: Bearer <token>.
En el backend, cada endpoint protegido valida el token y obtiene de él la identidad del usuario (ver sección de Backend). Por eso los servicios del cliente no envían el identificador del usuario como parámetro: solo adjuntan el token. Si el backend lo rechaza, el cliente recibe ErrorAutenticacion.

8.2.3.3 Registro de usuarios

Existen dos flujos de registro, uno por tipo de usuario:

- Postulante: POST /autenticacion/registrar-postulante. El cuerpo agrupa los datos en dos objetos: usuario (email, password) y postulante (nombre, apellido, ubicacion).
- Empleador: POST /autenticacion/registrar-empleador, con usuario (email, password) y empleador (nombre_negocio, ubicacion, direccion).

El cliente valida que los campos obligatorios estén completos, que el email contenga @, que la contraseña tenga al menos 8 caracteres y que coincida con su confirmación. Cada error se muestra debajo del campo correspondiente. El backend vuelve a validar los datos con Pydantic y, si los rechaza, el cliente recibe ErrorValidacion. Al registrarse correctamente, el usuario es enviado a la pantalla de inicio de sesión.

8.2.3.3 Recuperación de contraseña

El proceso tiene dos pasos. Primero el usuario ingresa su email y el cliente llama a POST /autenticacion/resetear-contraseña. Después, con el token de recuperación, el usuario define la nueva contraseña y el cliente llama a PUT /autenticacion/restablecer-contraseña?token=<token>, enviando nueva_contrasena en el cuerpo.

8.2.3.4 Expiración y cierre de sesión

Cuando el backend rechaza el token (por ejemplo, porque venció), ApiService lanza ErrorAutenticacion. Las vistas lo capturan, muestran el diálogo Sesión expirada y redirigen a /login. Las tareas periódicas y los WebSocket se detienen al abandonar la vista, para no seguir consultando con una sesión inválida.

9. IMPLEMENTACIÓN

9.1 Desarrollo de software

El desarrollo de LaburAR se llevó a cabo de manera progresiva, tomando como referencia los requerimientos definidos durante las etapas de análisis y diseño. La implementación se centró en construir las funcionalidades necesarias para la intermediación laboral entre postulantes y empleadores, contemplando también las tareas de administración y moderación de la plataforma.

El desarrollo se organizó en componentes diferenciados, con el propósito de separar las responsabilidades de acceso a los datos, procesamiento de la lógica de negocio y atención de las solicitudes realizadas por los usuarios.

Durante la implementación se desarrollaron las funcionalidades correspondientes a la autenticación y gestión de usuarios, los perfiles de postulantes y empleadores, la publicación y búsqueda de ofertas laborales, la gestión de postulaciones y las conexiones laborales. También se incorporaron las funciones de mensajería entre usuarios, administración de cuentas y gestión de reportes.

Asimismo, se integraron servicios externos para complementar determinadas funcionalidades de la aplicación, como el envío de correos electrónicos y la gestión de imágenes.

A medida que avanzó el desarrollo, se realizaron verificaciones sobre los distintos componentes para identificar errores, corregir inconsistencias y comprobar que el comportamiento de las funcionalidades se correspondiera con lo establecido en los requerimientos.

La implementación se acompañó con pruebas automatizadas, tanto unitarias como de integración, con el objetivo de evaluar los componentes individuales y su funcionamiento en conjunto.

El desarrollo se realizó de forma incremental, permitiendo incorporar funcionalidades, revisar su comportamiento y efectuar las correcciones necesarias durante el proceso.

9.2 Pruebas realizadas

108	133
PRUEBAS UNITARIAS APROBADAS	**PRUEBAS DE INTEGRACIÓN APROBADAS**

Durante el desarrollo de LaburAR se implementaron pruebas automatizadas con el propósito de verificar el comportamiento de las funcionalidades desarrolladas, detectar errores y comprobar el cumplimiento de las condiciones establecidas para cada operación.

Se utilizó pytest como herramienta de ejecución de pruebas y FastAPI TestClient para realizar solicitudes a los endpoints de la API durante las pruebas de integración.

Las pruebas se dividieron en dos categorías principales: pruebas unitarias y pruebas de integración.

9.2.1 Pruebas unitarias

Las pruebas unitarias se orientaron a verificar el comportamiento de componentes individuales de la aplicación, principalmente los servicios responsables de procesar la lógica de negocio.

Se evaluaron diferentes situaciones de funcionamiento, incluyendo operaciones exitosas, validaciones, búsquedas de registros y respuestas ante condiciones de error. También se revisaron las interacciones con dependencias externas cuando correspondía.

Aspecto	Descripción
Objetivo	Verificar el comportamiento de los componentes individuales de la aplicación.
Herramienta utilizada	pytest.
Componentes evaluados	Servicios de autenticación, postulantes, empleadores, ofertas laborales, postulaciones, conexiones laborales, conversaciones, mensajes, reportes y servicios externos.
Verificaciones realizadas	Validación de operaciones, respuestas esperadas, manejo de errores e interacciones con dependencias.
Pruebas aprobadas	108
Pruebas fallidas	0
Resultado	Todas las pruebas unitarias de la ejecución registrada finalizaron satisfactoriamente.

9.2.2 Pruebas de integración

Las pruebas de integración se utilizaron para comprobar el comportamiento de los endpoints de la API y la interacción entre las distintas capas de la aplicación.

Se realizaron solicitudes HTTP para verificar las respuestas del backend, los códigos de estado, los datos devueltos y el tratamiento de distintas situaciones. Las pruebas abarcaron funcionalidades de autenticación, gestión de postulantes y empleadores, ofertas laborales, postulaciones, conexiones laborales, conversaciones, mensajería, reportes y administración.

Para las funcionalidades que interactúan con servicios externos, se utilizaron simulaciones en las pruebas correspondientes, evitando depender del envío real de correos electrónicos durante esas verificaciones.

Aspecto	Descripción
Objetivo	Verificar la interacción entre los componentes del backend mediante solicitudes a la API.
Herramientas utilizadas	pytest y FastAPI TestClient.
Componentes evaluados	Autenticación, postulantes, empleadores, ofertas laborales, postulaciones, conexiones laborales, conversaciones, mensajes, administración y reportes.
Verificaciones realizadas	Códigos de respuesta HTTP, datos devueltos, validaciones y comportamiento de los endpoints.
Pruebas aprobadas	133
Pruebas fallidas	0
Resultado	Todas las pruebas de integración finalizaron satisfactoriamente, verificando el comportamiento de los endpoints evaluados y la interacción entre los componentes del backend.

10. CONCLUSIÓN

10.1 Resultados obtenidos

El desarrollo de LaburAR permitió construir una aplicación web orientada a la intermediación laboral a nivel local, con el propósito de facilitar el contacto entre personas que buscan empleo y empleadores que necesitan incorporar personal.

La aplicación contempla tres tipos de usuarios: postulantes, empleadores y administradores, cada uno con funcionalidades específicas según su rol. Los postulantes pueden gestionar su perfil profesional, cargar su currículum, buscar ofertas laborales, realizar postulaciones y consultar sus solicitudes de contacto. Por su parte, los empleadores pueden administrar su perfil, publicar y gestionar ofertas laborales, buscar postulantes, gestionar las postulaciones recibidas y enviar solicitudes de contacto.

Además, se incorporó un sistema de mensajería que permite la comunicación entre postulantes y empleadores, con la posibilidad de localizar conversaciones según el empleador, el postulante o la oferta laboral relacionada. La aplicación también contempla herramientas administrativas para gestionar usuarios y atender los reportes realizados dentro de la plataforma.

La definición de los requerimientos funcionales permitió organizar las capacidades previstas para cada tipo de usuario y establecer una base para el desarrollo. Asimismo, las entrevistas realizadas a comercios locales contribuyeron a identificar dificultades vinculadas con la búsqueda y difusión de oportunidades laborales, como la dispersión de la información entre distintos medios y el alcance limitado de algunas publicaciones.

Finalmente, se realizaron pruebas automatizadas para verificar el funcionamiento de los componentes desarrollados y detectar errores durante el proceso. De esta manera, LaburAR constituye una solución informática orientada a centralizar las oportunidades laborales locales y facilitar la interacción entre los actores involucrados.

10.2 Mejoras futuras

A partir del desarrollo realizado, se identifican distintas posibilidades de mejora que podrían incorporarse en futuras versiones de LaburAR para ampliar sus funcionalidades y mejorar la experiencia de uso.

Entre las principales mejoras propuestas se encuentran:

- **Ampliación del alcance geográfico:** incorporar progresivamente ofertas y empleadores de otras localidades cercanas.
- **Mejoras en la búsqueda laboral:** ampliar los criterios de búsqueda y filtrado para facilitar la localización de ofertas y postulantes según sus características.
- **Sistema de notificaciones:** incorporar avisos sobre nuevas postulaciones, solicitudes de contacto, cambios de estado y mensajes recibidos.

10.3 Reflexión final

El desarrollo de LaburAR representó una oportunidad para integrar los conocimientos adquiridos durante la formación como Analista en Sistemas y aplicarlos a un proyecto orientado a una problemática concreta.

A lo largo del proceso fue necesario analizar las necesidades de los potenciales usuarios, definir los requerimientos, organizar la estructura de la aplicación, desarrollar sus funcionalidades y verificar su funcionamiento mediante pruebas. Este recorrido permitió comprender la importancia de mantener una relación entre el problema identificado, los requerimientos establecidos y la solución implementada.

Las entrevistas realizadas a comercios locales aportaron una perspectiva más cercana a las dificultades relacionadas con la búsqueda y publicación de oportunidades laborales. A partir de esas experiencias, fue posible orientar la propuesta hacia una herramienta que reúna en un mismo espacio las ofertas laborales y facilite el contacto entre postulantes y empleadores.

Por otra parte, el desarrollo implicó resolver dificultades técnicas, revisar decisiones de implementación y comprobar el comportamiento de las funcionalidades mediante pruebas. Estas tareas contribuyeron a profundizar los conocimientos adquiridos y a reconocer la importancia de la organización, la validación y el mantenimiento durante la construcción de software.

En conclusión, LaburAR permitió abordar una necesidad del ámbito local mediante una propuesta tecnológica concreta. El proyecto constituye una base para continuar desarrollando una herramienta que contribuya a organizar la información laboral y simplificar el vínculo entre quienes buscan empleo y quienes necesitan incorporar personal.

11. BIBLIOGRAFÍA

- FastAPI. (s. f.). *Documentación oficial de FastAPI*. https://fastapi.tiangolo.com/
- SQLAlchemy. (s. f.). *Documentación oficial de SQLAlchemy*. https://docs.sqlalchemy.org/
- PostgreSQL Global Development Group. (s. f.). *Documentación oficial de PostgreSQL*. https://www.postgresql.org/docs/
- Flet. (s. f.). *Documentación oficial de Flet*. https://docs.flet.dev/
- Python Software Foundation. (s. f.). *Documentación oficial de Python*. https://docs.python.org/3/
- pytest. (s. f.). *Documentación oficial de pytest*. https://docs.pytest.org/
- FastAPI. (s. f.). *OAuth2 con contraseñas y JWT (JSON Web Tokens)*. https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/